STQA - UNIT 3: Test Design Techniques & Levels of Testing

END-SEM EXAM NOTES

How to use: every answer is for about **6 marks** (merged answers 6-9). Each answer starts with the **questions it covers**, then a **yellow box** (basic idea - only to understand, do not write). Everything after the green **ANSWER** line is what you write, including the figure. The green **Final Takeaway** box is for quick revision. **Blue box** = hard words in simple language. Prepared from the standard STQA / ISTQB-style syllabus (no reference PDF was supplied), so match the wording with your class notes.

No.	Topic	Freq	No.	Topic	Freq
1	Static techniques	3	8	Performance testing	2
2	Statement and branch coverage	3	9	Non-functional testing (and vs functional)	2
3	Path and condition coverage	3	10	White box, black box and grey box testing	2
4	Loop coverage testing	3	11	Experience-based techniques	2
5	Unit and integration testing	2	12	Dynamic test design techniques	1
6	System and acceptance testing	2	13	Ad hoc testing	1
7	Regression testing	2			

1. Static techniques

Freq 3 | 6-9 marks

Questions covered by this answer:

Q1: What do you think about static techniques?

Q2: How are the different static test case design techniques useful for software testing?

BASIC IDEA - just to understand (no need to write)

Mistakes hide not only in the program but also in requirements, design documents and test cases. **Static testing** finds them by **reading and analysing** these items **without running the program**. It is like proof-reading an essay before printing 1000 copies, instead of finding the typo after printing.

Example: A requirement says "the system should be fast". A review spots that "fast" is not measurable - before anyone has written any code.

ANSWER - write this in the exam

Definition: Static techniques check work products (requirements, design, source code, test cases) **without executing the code**. They find defects early, when they are cheapest to fix. There are two kinds: **reviews** (done by people) and **static analysis** (done by tools).

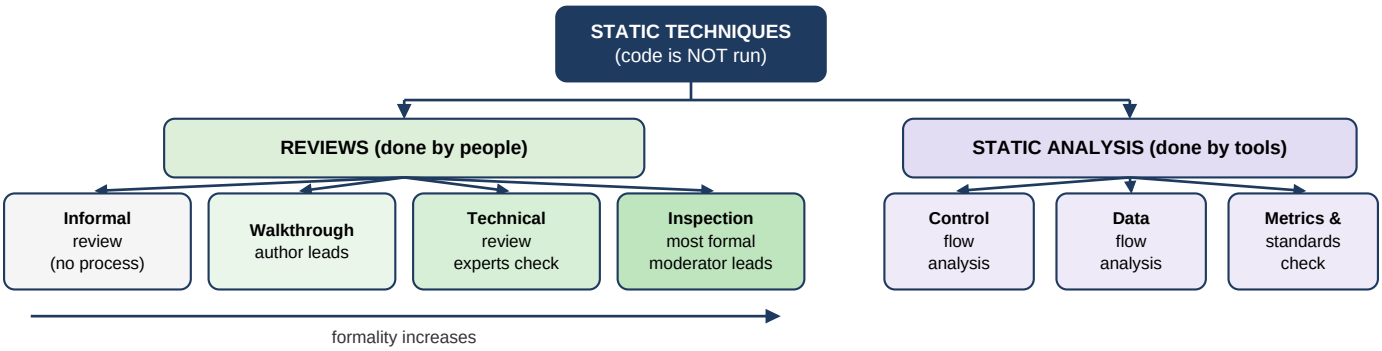


Fig 1.1: Static techniques

A. Reviews - types

Type	Led by	What happens / purpose
Informal review	Author or any colleague	No fixed process or documents; quick feedback from a colleague.
Walkthrough	Author	Author explains the document step by step to peers. Goal: learning, common understanding, finding defects. Open-ended.
Technical review	Trained moderator (not the author)	Peers / experts check technical correctness and standards; documented, may use checklists; ends with a decision.
Inspection	Trained moderator	Most formal. Defined roles (moderator, author, reader, scribe, inspectors), entry / exit criteria, checklists, metrics. Aim: find defects and improve the process.

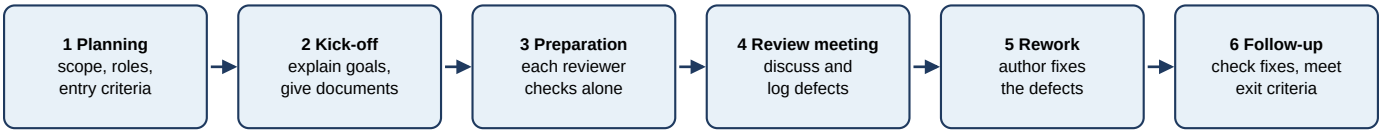


Fig 1.2: Steps of a formal review

B. Static analysis (tool-based)

A tool scans the source code **without running it** and reports: unreachable (dead) code, unused or uninitialised variables, undefined references, syntax and coding-standard violations, **data-flow anomalies** (a variable defined but never used, or used before it is defined), possible infinite loops, security weaknesses, and complexity (cyclomatic complexity). Examples of tools: compilers, Lint, PMD, Checkstyle, SonarQube.

C. How static techniques are useful

- **Early defect detection:** requirement and design defects are caught before coding; the cost of fixing grows in every later phase.
- **Finds the defect itself:** dynamic testing shows a failure and then needs debugging; a review points directly at the fault.
- **Finds what dynamic tests miss:** dead code, standard violations, unclear requirements, maintainability problems.
- **Better communication and quality:** team shares one understanding, reviewers learn good practice.
- **Less rework, test effort and time** in the rest of the project.
- **Limitations:** cannot find runtime or performance problems; reviews need time and skilled people; tools may give false alarms.

Final Takeaway: Static techniques = testing without running the code (reviews + tool analysis). They find defects earliest and cheapest, but must be combined with dynamic testing.

TERMS IN SIMPLE WORDS (write the ones you need)

Work product = any document or code made in the project

Moderator = person who leads a formal review

Dead code = code that can never be executed

Cyclomatic complexity = number of independent paths in code; higher = harder to test

Defect vs failure = defect = fault in code / document; failure = wrong behaviour seen when the program runs

2. Statement coverage and branch coverage

Freq 3 | 6 marks

Questions covered by this answer:

Q1: Explain statement coverage testing and branch coverage testing.

BASIC IDEA - just to understand (no need to write)

Coverage tells how much of the code your tests actually run. If a program has 100 statements and the tests run 80, coverage is 80%. Code that no test runs can hide bugs. Coverage testing is **white-box testing**: you look at the code to find which tests are still missing.

Example: Checking a house: **statement coverage** = you have walked into every room. **Branch coverage** = you have also used every door (open and closed outcome).

ANSWER - write this in the exam

Consider this small program (used for both techniques):

```
1  read x
2  if (x > 0)
3      x = x + 1
4  print x
```

1. Statement coverage (line coverage): every executable statement is executed **at least once**.
Formula: (statements executed / total statements) x 100.
Test x = 5 runs statements 1, 2, 3, 4 -> **100% statement coverage with just one test**. But the False case (x <= 0) was never tried, so a bug there stays hidden.

2. Branch coverage (decision coverage): every **outcome (True and False) of every decision** is executed at least once.
Formula: (decision outcomes executed / total outcomes) x 100.
Test x = 5 gives only the True outcome = 1 of 2 = **50%**. Adding test x = -1 (False outcome, skips statement 3) gives **100%**. So 2 tests are needed.

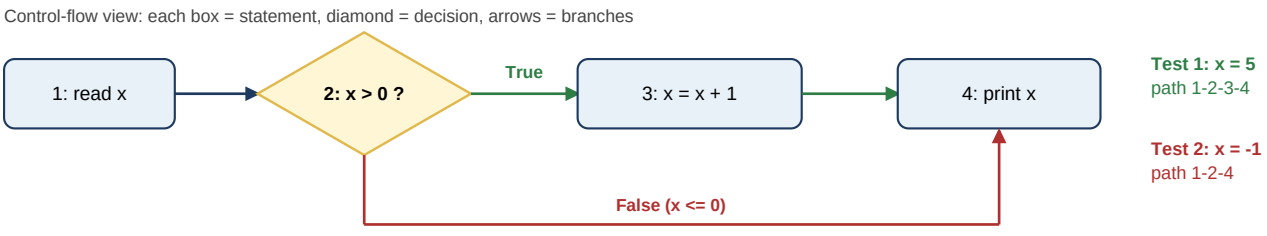


Fig 2.1: Control-flow view of the program with the two tests

Basis	Statement coverage	Branch coverage
Checks	Every statement runs once	Every True / False outcome of every decision runs once
Tests for the example	1 test (x = 5) gives 100%	2 tests (x = 5 and x = -1) needed
Strength	Weakest; ignores the missing False path	Stronger: 100% branch coverage always gives 100% statement coverage (not the reverse)

Basis	Statement coverage	Branch coverage
Weakness	Can miss bugs in the False / else path	Does not test combinations of conditions inside one decision

Final Takeaway: Statement coverage = every line runs; branch coverage = every True/False outcome runs. Branch coverage is stronger because it includes statement coverage.

TERMS IN SIMPLE WORDS (write the ones you need)

Statement = one executable line of code
Branch = one outcome (True or False) of a decision
Decision = point where the program chooses a path (if, while, switch)
Coverage = percentage of code elements exercised by tests

3. Path coverage and condition coverage

Freq 3 | 6 marks

Questions covered by this answer:
Q2: Explain path coverage testing and conditional coverage testing.

BASIC IDEA - just to understand (no need to write)

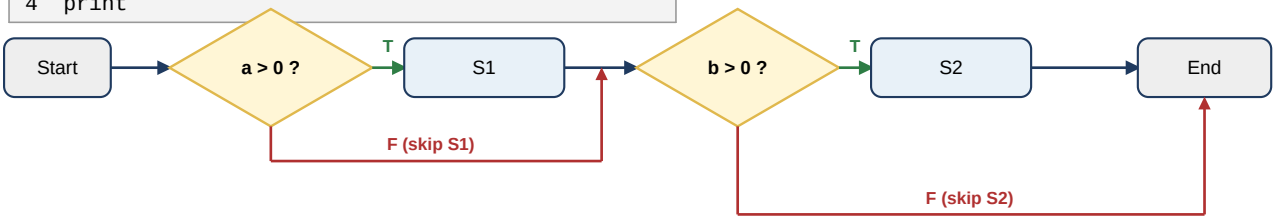
A **path** is one complete route a test takes through the program, from start to end. A **condition** is one small True/False test inside a decision: in (a > 0 AND b > 0) there are **two conditions** but only **one decision**.

Example: Path = a full journey from home to office by one particular road. With two junctions and two choices at each, there are 2 x 2 = 4 different journeys.

ANSWER - write this in the exam

1. Path coverage: every possible path from entry to exit is executed **at least once**. It is the strongest structural coverage. **Formula:** (paths executed / total paths) x 100. Example program:

```
1  read a, b
2  if (a > 0)  S1
3  if (b > 0)  S2
4  print
```



4 paths: (T,T) (T,F) (F,T) (F,F) = 2 x 2

Branch coverage needs only 2 tests (T,T) and (F,F); path coverage needs all 4.

Fig 3.1: Two decisions in sequence give four paths

Path	a > 0	b > 0	Statements executed
P1	True	True	1, 2, S1, 3, S2, 4
P2	True	False	1, 2, S1, 3, 4
P3	False	True	1, 2, 3, S2, 4
P4	False	False	1, 2, 3, 4

Problem: n decisions can give 2^n paths, and loops make paths unlimited. So in practice only **independent paths** are tested (**basis path testing**). Their count is the **cyclomatic complexity** V(G) = E - N + 2 = (number of decisions) + 1. Here V(G) = 2 + 1 = **3**.

2. Condition coverage: every **individual (atomic) condition** of a decision is evaluated to **both True and False** at least once. Example decision: if (A AND B) where A: a > 0 and B: b > 0.

Test	A	B	A AND B (decision)	Note
T1	True	False	False	A = T, B = F
T2	False	True	False	A = F, B = T
T3 (extra)	True	True	True	needed to also get the decision True

T1 and T2 give 100% **condition coverage** (A and B both took True and False), yet the decision was always False - so condition coverage **does not guarantee branch coverage**. Using both gives **decision/condition coverage**. Stronger forms: **multiple-condition coverage** (all 2^n combinations, 4 for two conditions) and **MC/DC** (each condition is shown to change the outcome independently, n + 1 tests; used in aviation software).

Basis	Path coverage	Condition coverage
Counts	Complete routes from start to end	True / False values of each small condition
Strength	Very strong but too many tests	Looks inside a decision; not enough alone
Practical use	Test independent paths using V(G)	Combine with branch coverage (decision/condition)

Final Takeaway: Path coverage tests every route from start to end (use V(G) independent paths in practice); condition coverage tests each True/False part inside a decision, and should be combined with branch coverage.

TERMS IN SIMPLE WORDS (write the ones you need)

Path = one full route from entry to exit of the program

Atomic condition = smallest True/False expression with no AND / OR inside

Independent path = a path that adds at least one new edge

Cyclomatic complexity = V(G) = E - N + 2 = decisions + 1

4. Loop coverage testing and its types

Freq 3 | 6 marks

Questions covered by this answer:

Q3: Discuss loop coverage testing and its types in detail.

BASIC IDEA - just to understand (no need to write)

A **loop** repeats code many times, and bugs love loop boundaries: the loop never runs, runs once, runs one time too many (**off-by-one** error) or never stops. **Loop testing** is a white-box technique that checks the validity of loop constructs by running each loop 0, 1 and many times.

Example: Testing a lift: carry 0 people, 1 person, a normal load, the maximum load, and one person above the maximum.

ANSWER - write this in the exam

Definition: Loop coverage means every loop in the program is executed **zero times, exactly once, and more than once** (typical and maximum). There are **four types** of loops and each is tested differently:

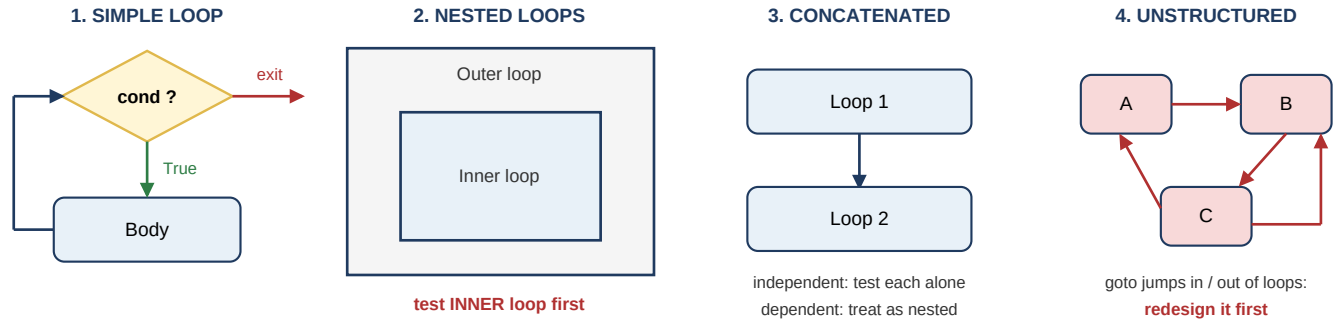


Fig 4.1: Types of loops

1. Simple loops (maximum n passes)

Test	Passes through the loop	What it checks
1	0 (skip the loop completely)	Loop condition false at start
2	1 pass	Basic working of the body
3	2 passes	Repetition works
4	m passes, where m < n (typical)	Normal operation
5	n-1, n and n+1 passes	Boundary and off-by-one errors

Example: for (i = 1; i <= 10; i++) sum = sum + i; -> test 0, 1, 2, 5, 9, 10 passes (and 11 if the input allows it).

2. Nested loops (one loop inside another)

- Start with the **innermost loop**; keep all outer loops at their **minimum** values.
- Test the inner loop as a simple loop (min, min+1, typical, max-1, max).
- Move one level outward: test it as a simple loop while the loops inside it stay at **typical** values and loops outside stay at minimum.
- Continue until all loops are tested. This avoids the huge number of tests (n^k) from trying all combinations.

3. Concatenated loops (loops one after another)

If the loops are **independent** (the count of one does not affect the other), test each as a simple loop. If they are **dependent** (the counter or result of the first loop is used by the second), treat them as **nested** loops.

4. Unstructured loops (goto jumping in and out)

These cannot be tested properly because control jumps into or out of the middle of loops. **Redesign them as structured loops first**, then test.

Final Takeaway: Loop testing runs each loop 0, 1, many and boundary times. Simple loops: 0, 1, 2, m, n-1, n, n+1; nested: innermost first; concatenated: independent = separate, dependent = nested; unstructured: rewrite first.

TERMS IN SIMPLE WORDS (write the ones you need)

Pass / iteration = one execution of the loop body

Off-by-one error = loop runs one time too many or too few

Structured loop = loop with one entry and one exit (for, while)

Infinite loop = loop whose exit condition never becomes true

5. Unit testing and integration testing

Freq 2 | 6-9 marks

Questions covered by this answer:
Q1: What do you mean by unit and integration testing? What are the approaches used in integration testing?

BASIC IDEA - just to understand (no need to write)

Software is built from many small pieces called **units** or **modules**. First test each piece alone (**unit testing**); then join the pieces and check that they work together (**integration testing**). It is like testing each Lego brick, then checking that the bricks snap together.

Example: Online shop: unit test = the "calculate total" function alone. Integration test = the cart module passes the correct total to the payment module.

ANSWER - write this in the exam

1. Unit testing: testing the **smallest testable part** (function, method, class or module) **in isolation**. Done by the **developer**, mostly **white-box**, with tools such as JUnit or NUnit. Missing neighbours are replaced by **stubs** (dummy called modules) and **drivers** (dummy calling programs). It checks logic, boundary values, error handling and data structures. *Advantages:* bugs found early and cheaply, easy debugging, safe code changes.

2. Integration testing: after unit testing, the units are **combined** and the **interfaces and data flow between them** are tested. It finds wrong data formats, mismatched parameters, missing calls and timing problems that single-unit tests cannot see.

Approaches to integration testing

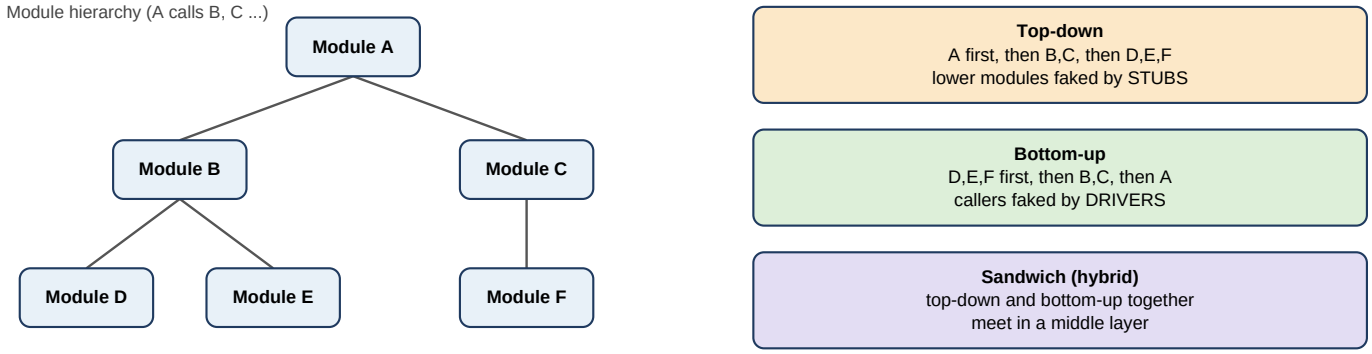


Fig 5.1: Module hierarchy and the integration orders

Approach	How it works	Pros	Cons
Big Bang	All modules joined at once and tested together; no stubs or drivers.	Simple; little planning.	Hard to find the faulty module; late defect detection; risky.
Top-down	Start from the top control module, add lower modules step by step; uses **stubs**.	Early working skeleton; design flaws found early.	Many stubs needed; lower modules tested late.
Bottom-up	Start from the lowest modules, join into clusters, move upward; uses **drivers**.	No stubs; easy fault location; low-level modules well tested.	Top-level logic tested late; no working system until the end.
Sandwich (hybrid)	Top-down and bottom-up together, meeting in a middle layer.	Faster, parallel work.	More complex and costly.

Basis	Unit testing	Integration testing
Tests	One module alone	Interfaces between modules
Done by	Developer	Developer / tester
Needs	Stubs and drivers	Modules combined (stubs / drivers in incremental approaches)
Finds	Logic and coding errors	Interface and data-flow errors

Final Takeaway: Unit testing checks one module alone; integration testing checks modules working together. Approaches: Big Bang, Top-down (stubs), Bottom-up (drivers), Sandwich.

TERMS IN SIMPLE WORDS (write the ones you need)

Module / unit = smallest independent part of the software
Stub = dummy lower module that returns fixed values
Driver = dummy upper module that calls the module under test
Interface = the point where two modules exchange data

6. System testing and acceptance testing

Freq 2 | 6 marks

Questions covered by this answer:
Q1: How would you explain system testing and acceptance testing?

BASIC IDEA - just to understand (no need to write)

After all modules are joined, the **whole product** is tested like a real customer would use it - this is **system testing**. Last of all, the **customer or user** checks whether the product really suits their needs and decides to accept it - this is **acceptance testing**.

Example: Buying a car: the factory test-drives the complete car (system testing); then the buyer test-drives it before taking delivery (acceptance testing).

ANSWER - write this in the exam

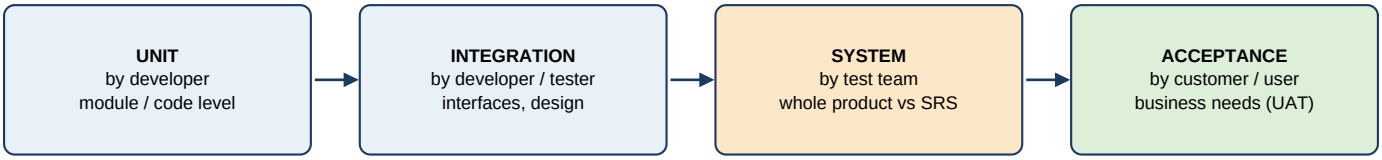


Fig 6.1: Levels of testing - system and acceptance are the last two

System testing: testing the **complete, integrated system** against the **requirements (SRS)** in an environment similar to production. It is **black-box** testing done by an independent test team, and covers both functional and non-functional behaviour: performance, security, usability, compatibility, recovery. It checks that the product is built right (verification against the specification).

Acceptance testing: the **final level**, done by the **customer / end user / business** to confirm the system meets business needs and is ready for delivery (validation: is it the right product?). Types:

- **User Acceptance Testing (UAT):** real users run real business scenarios.
- **Alpha testing:** at the developer site by internal staff or a separate team.
- **Beta testing:** at the customer site by real users in the real environment; feedback is collected.
- **Operational acceptance:** backup, recovery, maintenance and security procedures work.
- **Contract / regulatory acceptance:** checked against contract terms or legal standards.

Basis	System testing	Acceptance testing
Purpose	Verify the complete system against the SRS	Validate it meets user and business needs; decide to accept
Done by	Independent test team	Customer, end users, product owner
Based on	Functional and non-functional requirements	User requirements, business processes, contract
Environment	Test environment like production	Production-like or the customer site
Stage	After integration testing	Last stage before release

Example (online shopping app): System testing - login, search, cart, payment, speed with many users, security. Acceptance testing - the shop owner places real orders, checks invoices and approves the app for go-live.

Final Takeaway: System testing = test team checks the whole product against the SRS; acceptance testing = customer checks it meets their needs (UAT, alpha, beta) before accepting it.

TERMS IN SIMPLE WORDS (write the ones you need)

SRS = Software Requirements Specification, the agreed list of requirements

Verification = are we building the product right (matches the specification)

Validation = are we building the right product (meets user needs)

End user = the person who finally uses the software

7. Regression testing

Freq 2 | 6 marks

Questions covered by this answer:
Q1: Identify the importance of regression testing and explain it.

BASIC IDEA - just to understand (no need to write)

Whenever software is changed (a bug fixed, a feature added), the change can **accidentally break something that worked before**. **Regression testing** means running the old tests again to make sure the old features still work.

Example: A mechanic fixes your brakes and then also checks that the lights, horn and engine still work - the repair must not damage anything else.

ANSWER - write this in the exam

Definition: Regression testing is the **re-testing of already tested software after a change** to confirm that the change has not introduced new defects (side effects) in unchanged parts. It is done after a bug fix, new feature, configuration change or environment change.

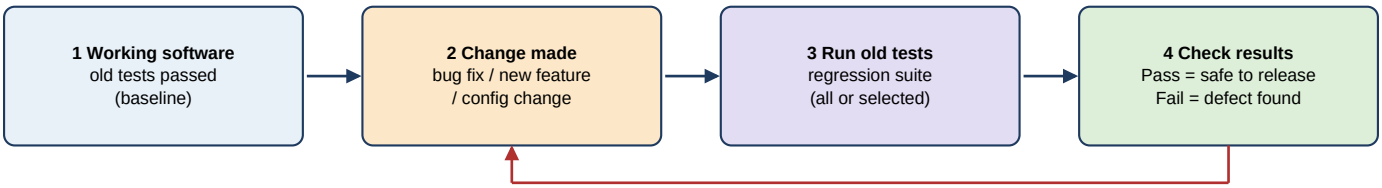


Fig 7.1: Regression testing cycle

Importance

- Makes sure **fixes and new features do not break existing functionality**.
- Finds **side effects** early, when they are cheap to fix.
- Keeps **quality stable across releases**; builds confidence before release.
- Essential in **agile / continuous integration** where code changes very often.
- Reduces production failures and customer complaints; regression tests are good candidates for **automation** (Selenium, JUnit).

Types / techniques

Type	Meaning
Retest-all (complete)	Run the entire old test suite; safest but slow and costly.
Selective (selection)	Run only tests related to the changed and affected modules.
Prioritised	Run the most important / high-risk tests first.
Corrective vs progressive	Corrective: no change in requirements, reuse existing tests. Progressive: requirements changed, new test cases are added.

Regression vs retesting: *retesting* re-runs the failed test to confirm the specific bug is fixed; *regression testing* checks that nothing else is broken.

Example: A bug in the discount calculation of a shopping app is fixed. Regression tests re-check cart total, tax, invoice and payment.

Final Takeaway: Regression testing re-runs old tests after every change to prove that nothing that worked before is broken; it is vital for stable releases and is best automated.

TERMS IN SIMPLE WORDS (write the ones you need)

Side effect = unwanted effect of a change on another part
Baseline = the last known good version
Test suite = a collection of test cases run together
Automation = tests run by a tool instead of manually

8. Performance testing with an example

Freq 2 | 6 marks

Questions covered by this answer:
Q1: Explain performance testing with an example.

BASIC IDEA - just to understand (no need to write)

A program may give correct answers and still be **too slow** or **crash when many people use it at once**. **Performance testing** checks how fast, stable and scalable the software is under a given workload.

Example: Checking a bus: how fast does it go, how many passengers can it carry, and what happens when too many climb in?

ANSWER - write this in the exam

Definition: Performance testing is **non-functional testing** that measures speed (response time), throughput, stability, scalability and resource use (CPU, memory) under expected and extreme load, to find **bottlenecks** before users do.

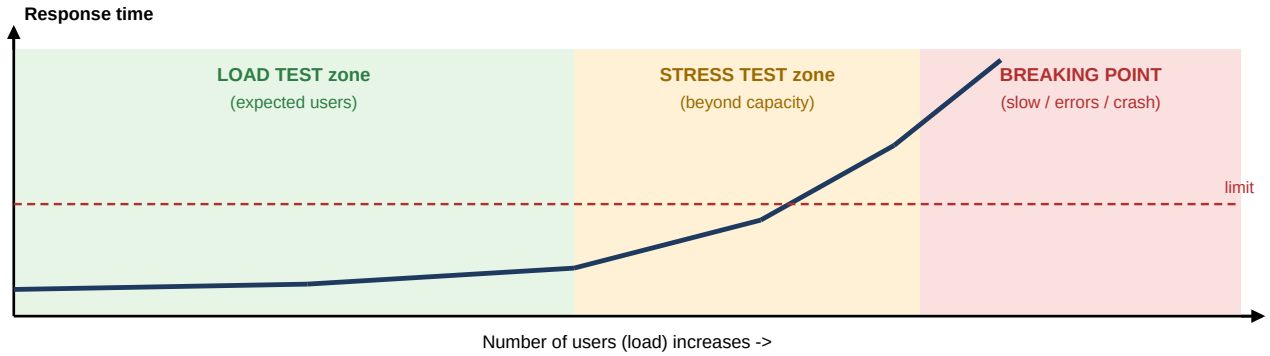


Fig 8.1: Response time rises with load; stress testing finds the breaking point

Type	What it does
Load testing	Checks behaviour under the **expected** number of users.
Stress testing	Pushes the system **beyond capacity** to find the breaking point and see how it recovers.

Type	What it does
Spike testing	Sudden sharp rise or drop in users (e.g. flash sale).
Endurance (soak) testing	Normal load for a **long time** to find memory leaks and slow-downs.
Volume testing	Very **large amounts of data** in the database or files.
Scalability testing	Checks how well performance grows when resources / users are increased.

Steps: 1. Set goals and criteria (e.g. response time < 3 s) -> 2. Prepare a test environment like production -> 3. Design scenarios -> 4. Run with a tool (JMeter, LoadRunner, Gatling) -> 5. Analyse results and tune -> 6. Retest. **Metrics:** response time, throughput, concurrent users, error rate, CPU / memory use.

Example: An online ticket-booking site expects 10,000 users on sale day with response time under 3 seconds. A **load test** simulates 10,000 virtual users; a **stress test** raises it to 50,000 to see where pages slow or fail. The team then fixes the bottleneck (slow database query, small server) and retests.

Final Takeaway: Performance testing measures speed, stability and scalability under load (load, stress, spike, endurance, volume) to find bottlenecks before release.

TERMS IN SIMPLE WORDS (write the ones you need)

Response time = time taken to answer a request

Throughput = number of requests handled per second

Bottleneck = the slowest part that limits overall speed

Virtual users = simulated users created by the test tool

9. Non-functional testing (and functional vs non-functional)

Freq 2 | 6-9 marks

Questions covered by this answer:

Q1: Illustrate non-functional testing.

Q2: Differentiate between functional testing and non-functional testing.

BASIC IDEA - just to understand (no need to write)

Functional testing asks: *does the software do what it should?* (login works). **Non-functional testing** asks: *how well does it do it?* (fast, secure, easy to use). A car: driving is the function; speed, safety and comfort are the non-functional qualities.

Example: Login: functional = a valid user and password gets in. Non-functional = login finishes within 2 seconds even with 5000 users at once.

ANSWER - write this in the exam

Definition: Non-functional testing checks the **quality attributes** of the system that are not about specific features: how well it performs, how safe, usable and reliable it is.

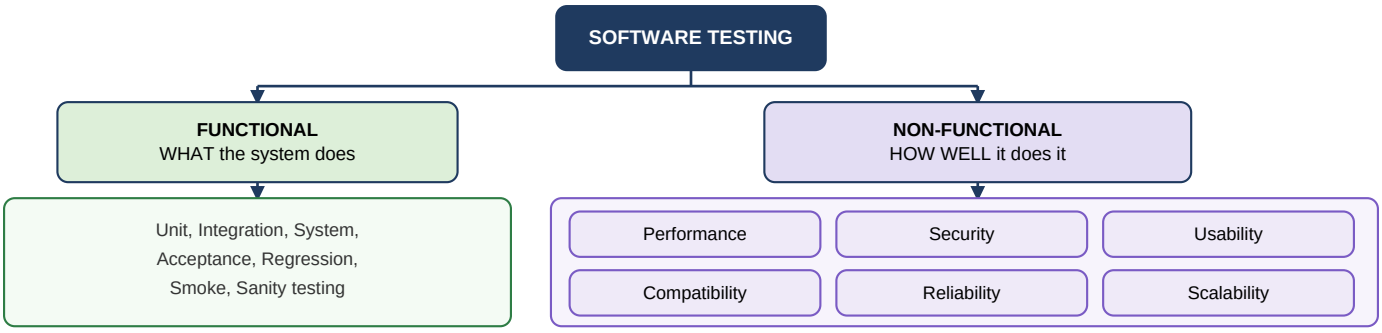


Fig 9.1: Functional and non-functional testing

Type	What it checks	Example
Performance	Speed, load, stability	Page loads in under 3 s for 1000 users
Security	Protection from unauthorised access and attacks	SQL injection and password strength tests
Usability	Ease of learning and use	New user finishes checkout without help
Compatibility	Works on different browsers, OS, devices	Same site on Chrome, Firefox, Android, iOS
Reliability	Failure-free operation over time	App runs 24 hours without a crash
Scalability / Recovery	Grows with load; recovers after failure	Restart after power cut with no data loss

Functional vs non-functional testing

Basis	Functional testing	Non-functional testing
Checks	WHAT the system does (features)	HOW WELL it does it (quality)
Based on	Functional / business requirements	Non-functional requirements (speed, security ...)
Focus	Correct output for given input	Performance, usability, reliability, security
Examples	Unit, integration, system, UAT, smoke, regression	Performance, load, stress, security, usability
Result	Pass / fail against expected output	Measured against limits (e.g. under 3 s)
Execution	Usually done first; manual or automated	Usually after functional testing; often needs tools

Final Takeaway: Functional testing checks WHAT the software does; non-functional testing checks HOW WELL it does it (performance, security, usability, compatibility, reliability).

TERMS IN SIMPLE WORDS (write the ones you need)

Quality attribute = a quality like speed or security of the system

Requirement = what the customer wants from the software

SQL injection = attack that inserts harmful database commands through input fields

Compatibility = ability to work in different environments

10. White box, black box and grey box testing

Freq 2 | 6-9 marks

Questions covered by this answer:

Q1: Differentiate between black box and white box testing.

Q2: Explain white box testing and grey box testing in detail.

BASIC IDEA - just to understand (no need to write)

Imagine the software as a **sealed box**. In **black-box** testing you only give inputs and check outputs. In **white-box** testing the box is transparent and you see the code. In **grey-box** testing you can see only a part, such as the database or design.

Example: Login page: black box = try valid and invalid passwords. White box = check every if-branch of the validation code. Grey box = knowing the database table, check that the session record is created after login.

ANSWER - write this in the exam

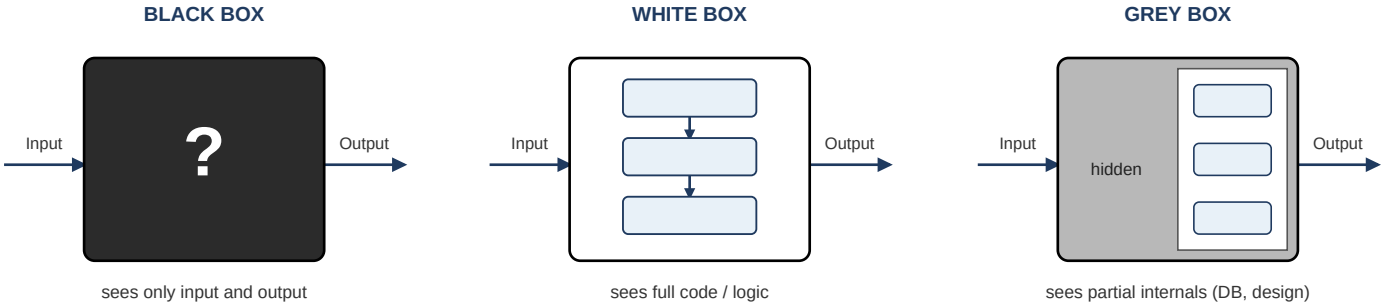


Fig 10.1: Black box, white box and grey box - what the tester can see

White box testing (glass box / structural / code-based): the tester knows the **internal code and logic** and designs tests from it. Done by developers. **Techniques:** statement, branch, path, condition and loop coverage, data-flow testing.
Advantages: thorough, finds hidden logic errors, removes dead code, helps optimisation. *Disadvantages:* needs programming skill, costly, cannot find missing features, tests break when code changes.

Black box testing (functional / behavioural): tests from the **requirements only**, no code knowledge. Done by testers. **Techniques:** equivalence partitioning, boundary value analysis, decision table, state transition, use case testing. *Advantages:* user viewpoint, no programming needed. *Disadvantages:* limited coverage, may miss hidden code paths.

Grey box testing: a **mix** - the tester has **partial knowledge** of internals (database structure, architecture, APIs, algorithms) but tests from outside, like a user. **Techniques:** matrix testing, regression testing, pattern testing, orthogonal array testing. Used in web, database, integration and security (penetration) testing.
Advantages: better tests than black box without needing full source code. *Disadvantages:* partial coverage, cannot test all paths.

Basis	Black box	White box	Grey box
Knowledge of code	None	Full	Partial
Also called	Functional / behavioural	Glass box / structural	Translucent
Based on	Requirements (SRS)	Code and design	Requirements + some design / DB knowledge
Done by	Testers	Developers	Testers / developers
Programming skill	Not needed	Required	Some needed
Main techniques	EP, BVA, decision table, state transition	Statement, branch, path, condition, loop coverage	Matrix, pattern, orthogonal array
Mostly used at	System, acceptance level	Unit level	Integration level
Coverage / time	Limited; fast	Highest; slow	Moderate

Final Takeaway: Black box = tests by requirements, no code view; white box = tests by code, full view; grey box = partial view of internals with a user-side test approach.

TERMS IN SIMPLE WORDS (write the ones you need)

Internal structure = the code, logic and design inside the software

Penetration testing = security test that tries to break in like an attacker

Orthogonal array = table that picks a small set of input combinations covering all pairs

API = interface through which one program talks to another

11. Experience-based techniques (error guessing, exploratory testing)

Freq 2 | 6 marks

Questions covered by this answer:

Q1: Write a brief outline of experience-based techniques.

Q2: State in your own words error guessing and exploratory testing.

BASIC IDEA - just to understand (no need to write)

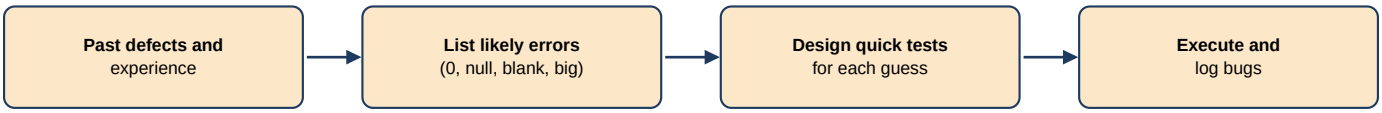
Sometimes testers do not follow strict documents but use their **experience, skill and intuition** to decide what to test, like a mechanic who listens to the engine noise and guesses the fault. These techniques are useful when specifications are missing or time is short.

Example: A tester who has seen many forms before immediately tries leaving fields blank, typing 0, or pasting a very long text - because such inputs broke other forms.

ANSWER - write this in the exam

Experience-based techniques derive tests from the tester's **knowledge, past experience with similar applications, and knowledge of common defects**, instead of from formal rules. They **complement** black-box and white-box techniques. Main techniques: **error guessing**, **exploratory testing** and **checklist-based testing**.

ERROR GUESSING



EXPLORATORY TESTING (learn + design + execute happen together)

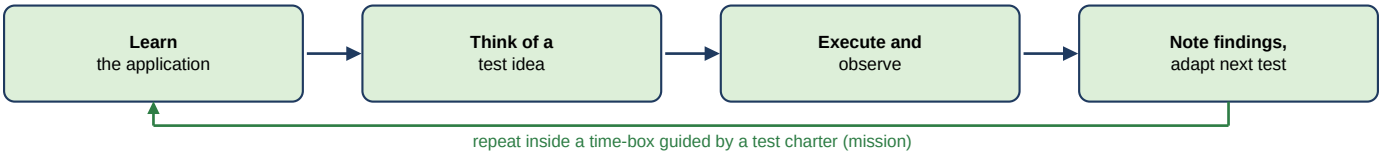


Fig 11.1: Error guessing and exploratory testing

1. Error guessing: the tester **guesses where errors are likely** using past defects and experience, makes a **list of possible errors** (also called a fault attack) and designs tests for them. Typical guesses: divide by zero, empty or null input, blank mandatory fields, special characters, maximum length, invalid dates (30 Feb), negative numbers, duplicate entries, wrong file type.
Pros: quick, cheap, finds defects formal techniques miss. *Cons:* depends on tester skill, not repeatable, coverage cannot be measured.

2. Exploratory testing: learning, test design and test execution happen at the same time. The tester explores the application, uses what is learned to choose the next test, and notes findings. It is guided by a **test charter** (a short mission) and a **time-box** (session-based testing, e.g. 60-90 minutes).
Pros: finds unexpected defects, adapts quickly, good when documents are poor. *Cons:* hard to measure coverage, depends on tester, less repeatable.

3. Checklist-based testing: the tester uses a list of items to check (built from experience), e.g. UI standards or common error types.

Basis	Error guessing	Exploratory testing
Idea	Guess likely errors and test them	Learn, design and execute together
Planning	List of guessed errors	Charter and time-box
Documentation	Very little	Session notes
Example	Enter 0 in quantity field	60-minute session: explore checkout with different payment methods

Final Takeaway: Experience-based techniques use the tester's skill and intuition: error guessing tests guessed weak spots; exploratory testing learns, designs and tests at the same time within a charter.

TERMS IN SIMPLE WORDS (write the ones you need)

Test charter = short mission statement for an exploratory session

Time-box = fixed time limit for a session

Fault attack = targeted tests built from a list of likely errors

Intuition = judgement gained from past experience

12. Dynamic test design techniques

Freq 1 | 6 marks

Questions covered by this answer:
Q1: Interpret the different dynamic test design techniques.

BASIC IDEA - just to understand (no need to write)

Dynamic testing means testing by **actually running** the software with inputs. We cannot try every possible input, so **dynamic test design techniques** are systematic ways to **choose the inputs** that find the most defects with the fewest tests.

Example: A form accepts ages 18 to 60. Instead of testing all numbers, you pick a few smart values: one valid, one too low, one too high, and the edge values.

ANSWER - write this in the exam

Definition: Dynamic test design techniques are methods used to design test cases that are **executed on the running software**. They are grouped into three families:

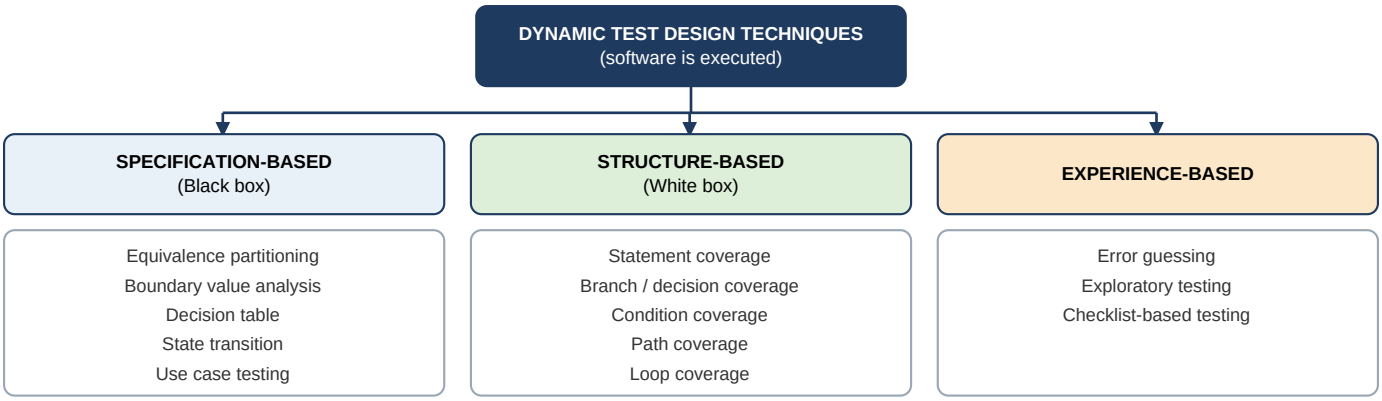
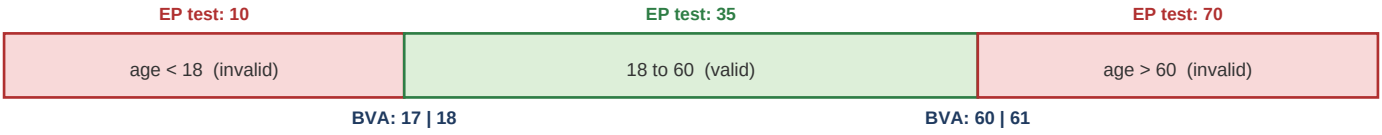


Fig 12.1: Dynamic test design techniques

Technique	Idea	Small example
Equivalence partitioning	Divide inputs into groups that behave the same; test one value per group.	Age 18-60: test 10, 35, 70
Boundary value analysis	Test at the edges of each group, where bugs are common.	Test 17, 18, 60, 61
Decision table	List combinations of conditions and the resulting actions.	Member? / Amount > 1000? -> discount
State transition	Test movement between states and events.	ATM: card in -> PIN -> menu -> cash out
Use case testing	Test the main and alternate flows of a use case.	Login: valid, wrong password, locked account
Structure-based (white box)	Cover statements, branches, conditions, paths, loops of the code.	Tests that run both outcomes of an if
Experience-based	Use tester skill: error guessing, exploratory, checklist.	Blank fields, 0, very long text

Example: an age field accepts values from 18 to 60



EP = one test from each box | BVA = tests at the edges of the boxes

Fig 12.2: Equivalence partitioning and boundary value analysis

Static vs dynamic: static techniques examine documents/code without running them; dynamic techniques need the software to run.

Final Takeaway: Dynamic techniques pick smart test inputs for running software: specification-based (EP, BVA, decision table, state transition, use case), structure-based (coverage) and experience-based.

TERMS IN SIMPLE WORDS (write the ones you need)

Equivalence class = group of inputs the program treats the same way
Boundary = the edge value between two groups
Decision table = table of condition combinations and actions
State = a situation of the system, e.g. "card inserted"

13. Ad hoc testing with an example

Freq 1 | 6 marks

Questions covered by this answer:
Q2: Illustrate adhoc testing with an example.

BASIC IDEA - just to understand (no need to write)

"Ad hoc" means **done for the moment, without a plan**. In ad hoc testing the tester tries the software **randomly and informally**, without test cases or documents, using knowledge of the application to try to break it. Like a child pressing random buttons on a remote.

Example: On a shopping app a tester adds an item, presses Back, changes quantity to -1 repeatedly, switches tabs quickly and logs out during payment - and finds a crash no scripted test covered.

ANSWER - write this in the exam

Definition: Ad hoc testing is an **informal, unplanned and unstructured** testing method with **no test plan, test cases or documentation**. It is usually done by an experienced tester **after formal testing** or when time is short, to find defects that scripted tests missed.

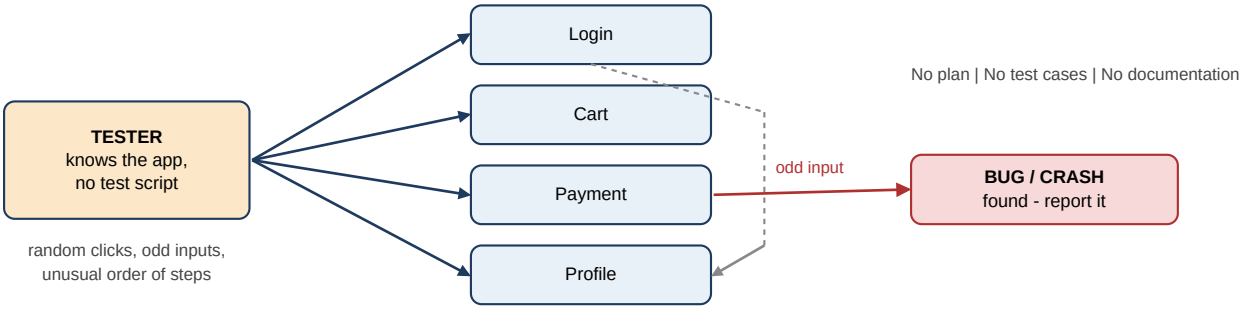


Fig 13.1: Ad hoc testing

Characteristics

- No planning, no documentation; tests are made up while testing.
- Needs good knowledge of the application and tester intuition.
- Done randomly, usually once; hard to repeat or measure.

Types

- **Buddy testing:** a developer and a tester work together on the same module.
- **Pair testing:** two testers share one machine; one tests, the other notes.
- **Monkey testing:** random inputs and actions to check whether the system crashes.

Advantages: quick, no preparation, finds unexpected defects. **Disadvantages:** no coverage measure, not repeatable, bugs are hard to reproduce, depends on tester skill.

Ad hoc vs exploratory: exploratory testing is structured (charter, time-box, notes); ad hoc testing has no structure at all.

Final Takeaway: Ad hoc testing = random, informal testing without plan or documents, done by an experienced tester to catch defects that planned tests miss (buddy, pair, monkey testing).

TERMS IN SIMPLE WORDS (write the ones you need)

Informal = not following a fixed written process
Monkey testing = giving random input like a monkey pressing keys
Reproduce = repeat the steps to see the same bug again
Test case = input + steps + expected result written for a test

STQA - UNIT 4: Quality Management, Standards & Defects

END-SEM EXAM NOTES

How to use: Every answer is for **6 to 9 marks** (write fewer parts for fewer marks). **Yellow box** = basic idea, only to understand (do not write). Everything after **ANSWER** = write in the exam (draw the figure too). **Blue box** = hard words explained simply - write the ones you need. Where an answer has **Part A / Part B**, each part is complete by itself: write only the part asked, or both if the question joins them. **Contents** (bracket = how many past questions one answer covers): Q1 Quality Management System + Quality Plan (4) | Q2 CMM levels and limitations (2) | Q3 SDLC steps + Waterfall model (2) | Q4 Why software has defects, problem areas of SDLC, impact of defects (3) | Q5 QA vs QC + Quality Control methods (2) | Q6 ISO 9001 (1) | Q7 Customer Satisfaction (2) | Q8 Software characteristics + Products by criticality (2) | Q9 Selenium IDE (1).

Find the right answer quickly:

If the question says...	Write
Quality management system / aspects of quality management / pillars of QMS	Q1 Part A
Quality plan	Q1 Part B
Levels of CMM / limitations of CMM	Q2 (levels + limitations are both inside)
Steps of software development / Waterfall model with diagram and limitations	Q3 Part A / Q3 Part B
Problematic areas in SDLC / Why does software have defects?	Q4 Part A (one answer serves both)
Impact of a defect in different phases	Q4 Part B
QA vs QC / Quality control and two methods	Q5 Part A / Q5 Part B
Characteristics of software / Types of products by criticality	Q8 Part A / Q8 Part B

Q1. Quality Management System (QMS) + Quality Plan

Covers 4 Qs | 6-9 marks

> ***This one answer covers these questions:*** *1) Can you clarify the quality management system? | 2) Explain the important aspects of quality management. (Part A) | 3) Explain in detail the pillars of the quality management system. (Part A) | 4) Can you explain the quality plan in detail? (Part B)*

BASIC IDEA - just to understand (no need to write)

Think of a good school. It is not just the exam at the end; it has **admission rules, teaching methods, trained teachers, regular checks and feedback**, all working together so every student gets good education. A **QMS is that whole system for a software company**: the rules, people, processes and checks that make sure every product meets the customer's needs - and keeps getting better.

Example: A software company has a rule that every module must be reviewed, tested, and its defects recorded and analysed. That complete set of rules and records is its QMS.

ANSWER - write this in the exam (draw the figure too)

Part A: Quality Management System

Definition: A **Quality Management System (QMS)** is a formal set of **policies, processes, procedures, responsibilities and resources** used by an organization to **consistently deliver products that meet customer and regulatory requirements** and to **continually improve**. Standards such as ISO 9001 describe it.

Pillars of QMS (draw)

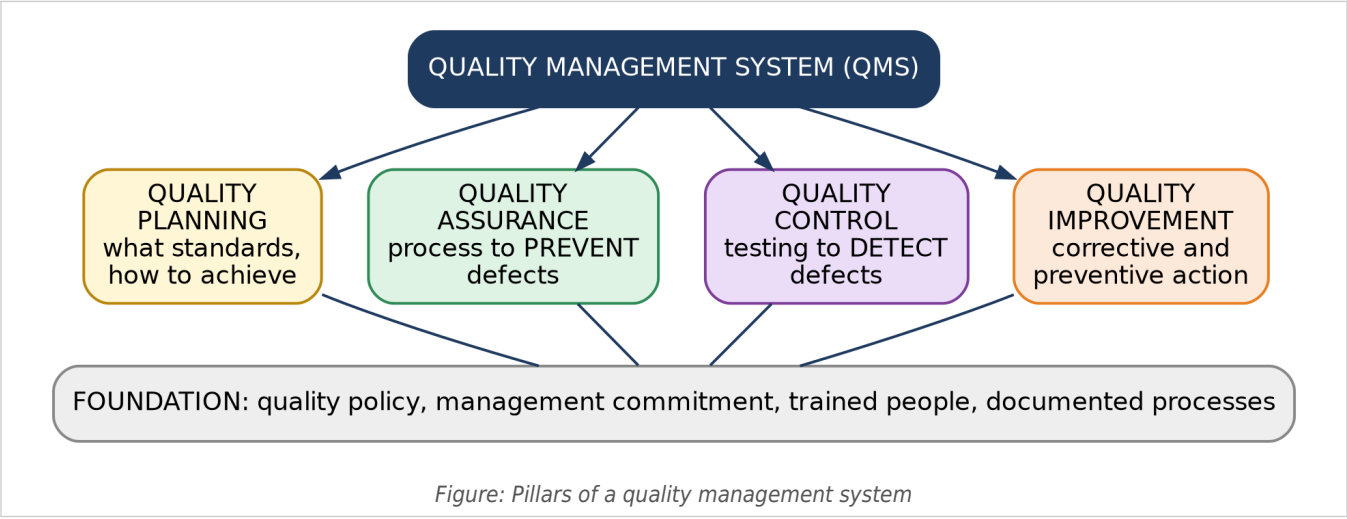


Figure: Pillars of a quality management system

- **Quality Planning:** decide quality objectives, standards and how they will be achieved (the quality plan).
- **Quality Assurance (QA):** process-based activities - standards, audits, reviews, training - that **prevent** defects.
- **Quality Control (QC):** product-based activities - inspection and testing - that **detect** defects.
- **Quality Improvement:** analyse defects and feedback, then apply corrective and preventive actions so the same problem does not repeat.
- **Foundation:** quality policy, top-management commitment, trained people and documented processes.

Important aspects of quality management

Aspect	Meaning
Customer focus	Understand customer needs and aim to meet or exceed them
Leadership	Management sets the quality policy and provides resources
People involvement	Trained, responsible team members at every level
Process approach	Manage work as defined, repeatable processes
Continuous improvement	PDCA cycle: Plan - Do - Check - Act
Fact-based decisions	Use metrics and data (defects, effort), not guesses
Prevention over detection	Avoid defects early rather than only finding them late
Documentation and records	Procedures, reviews and test records give evidence and traceability

Part B: Quality Plan

Definition: A **quality plan** is a document that states **how quality will be achieved for a particular project**: what standards apply, who does what, which activities (reviews, tests, audits) will be done, when, and how results are measured.

Content of quality plan	What it says
Quality objectives / scope	Targets, e.g. 'fewer than 0.5 defects per KLOC at release'
Standards and procedures	Coding standards, ISO 9001, templates, checklists to follow
Roles and responsibilities	Who reviews, who tests, who approves
Verification and validation plan	Reviews, inspections, test levels, test strategy
Metrics and reporting	Defect density, test coverage, review efficiency; how often reported
Tools and resources	Test tools, defect tracker, environments, training
Defect and change management	How defects are logged, fixed, retested; change control
Risks and acceptance criteria	Quality risks, exit criteria and conditions for delivery

HARD WORDS EXPLAINED - write the ones you need

Policy: a formal statement by management of the organization's intention about quality

Process: a defined series of steps to do a job

Audit: an independent check that the agreed process is really being followed

PDCA: Plan-Do-Check-Act: a repeating cycle for improvement

Traceability: ability to link requirements to design, code and tests

Corrective / preventive action: fix an existing problem / stop a possible problem

FINAL TAKEAWAY: A QMS is the complete system of planning, assuring, controlling and improving quality; the quality plan is its project-level document.

Q2. Capability Maturity Model (CMM): Levels and Limitations

Covers 2 Qs | 6-9 marks

This one answer covers these questions: *1) Can you clarify the different levels of CMM? | 2) List and explain the limitations of Capability Maturity Models (CMM).*

BASIC IDEA - just to understand (no need to write)

Imagine a cook. At first he cooks **by guess**, and the taste changes every day. Then he follows a **recipe**, then the **whole kitchen follows one standard recipe book**, then they **measure everything**, and finally they keep **improving the recipes**. CMM grades a software company's way of working in the same five stages.

Example: Level 1 company: success depends on one brilliant developer. Level 3 company: every project follows the same documented process, no matter who works on it.

ANSWER - write this in the exam (draw the figure too)

Definition: The **Capability Maturity Model (CMM)**, developed by the **Software Engineering Institute (SEI)** at Carnegie Mellon University, is a framework to **assess and improve the maturity of an organization's software development process** in five levels. (Its successor is CMMI.)

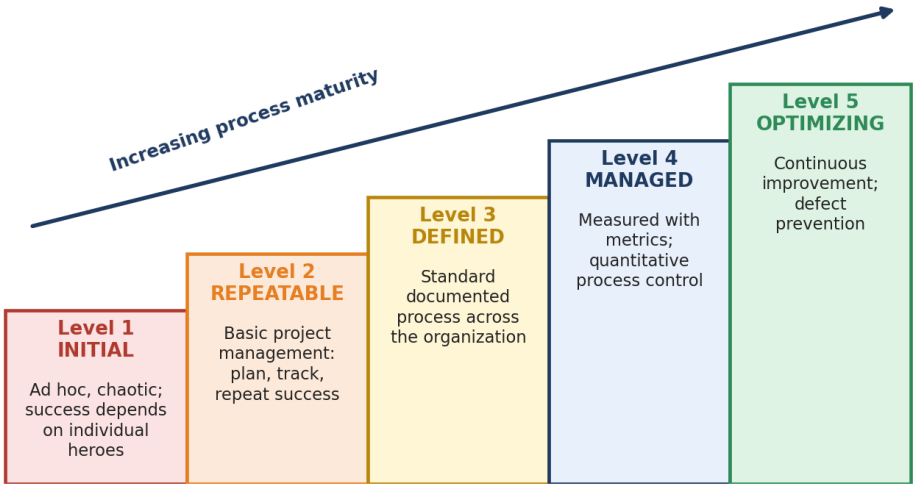


Figure: Five levels of CMM

Level	Characteristics	Key focus
1. Initial	Ad hoc, unpredictable, depends on individual effort	Nothing is defined
2. Repeatable	Basic project management; earlier success can be repeated on similar projects	Requirements management, planning, tracking, configuration management, QA
3. Defined	A standard, documented process used across the organization	Organization process definition, training, peer reviews
4. Managed	Process and product quality are measured and controlled using metrics	Quantitative process and quality management
5. Optimizing	Continuous improvement using feedback and new ideas	Defect prevention, technology and process change management

Note: in CMMI the level names are Initial, Managed, Defined, Quantitatively Managed, Optimizing.

Limitations of CMM

Limitation	Explanation
Heavy documentation	Too much paperwork and bureaucracy; slows real work
Costly and time-consuming	Assessment and training are expensive; hard for small companies
Process, not product	Good process does not guarantee good software
Ignores people and creativity	Focus on procedures, less on skills, motivation and innovation
Poor fit for agile / fast change	Rigid, plan-driven; not ideal for small or rapidly changing projects
Subjective assessment	Companies may 'show' compliance on paper without real practice
Slow progress	Moving up one level may take 1-3 years
One size fits all	Same model for all project types and sizes

HARD WORDS EXPLAINED - write the ones you need

SEI: Software Engineering Institute, the US research center that created CMM

Maturity: how well-defined, managed and improved a process is

Process: the steps an organization follows to build software

Metrics: measured numbers such as defects per module

Peer review: colleagues examine each other's work to find defects

Configuration management: controlling versions and changes of code and documents

FINAL TAKEAWAY: CMM grades a company's process from chaotic (Level 1) to continuously improving (Level 5), but it can be costly, document-heavy and slow.

Q3. SDLC Steps and the Waterfall Model

Covers 2 Qs | 6-9 marks

This one answer covers these questions: *1) Identify the different steps involved in the software development process. (Part A) | 2) With a neat diagram, discuss the waterfall model of software development. Also explain its limitations. (Part B)*

BASIC IDEA - just to understand (no need to write)

Building software is like **building a house**: first understand what the family wants, then draw the plan, then build, then inspect, then hand over, and later repair. SDLC is this ordered list of steps. The **waterfall model** does the steps strictly one after another, like water falling down steps - you do not go back up.

Example: A payroll system with fixed rules: collect requirements, design, code, test, deploy, then maintain. Fits waterfall because requirements will not change.

ANSWER - write this in the exam (draw the figure too)

Part A: Steps in the software development process (SDLC)

SDLC (Software Development Life Cycle) is the structured sequence of phases used to plan, create, test and deliver software.

Phase	What happens	Output
1. Planning / feasibility	Study cost, time, technical and business feasibility	Feasibility report, project plan
2. Requirement analysis	Collect and document what the user needs	SRS (Software Requirement Specification)
3. Design	Plan architecture, modules, database, interfaces	Design document (HLD / LLD)
4. Implementation (coding)	Developers write the code	Source code
5. Testing	Find and fix defects; verify and validate	Test reports, defect log
6. Deployment	Release to users / install	Working system
7. Maintenance	Fix bugs, add enhancements after release	Updates and patches

Part B: Waterfall model

Definition: The **Waterfall model** is a **linear, sequential** SDLC model in which each phase must be completed and approved before the next phase begins, with documentation as the output of each phase.

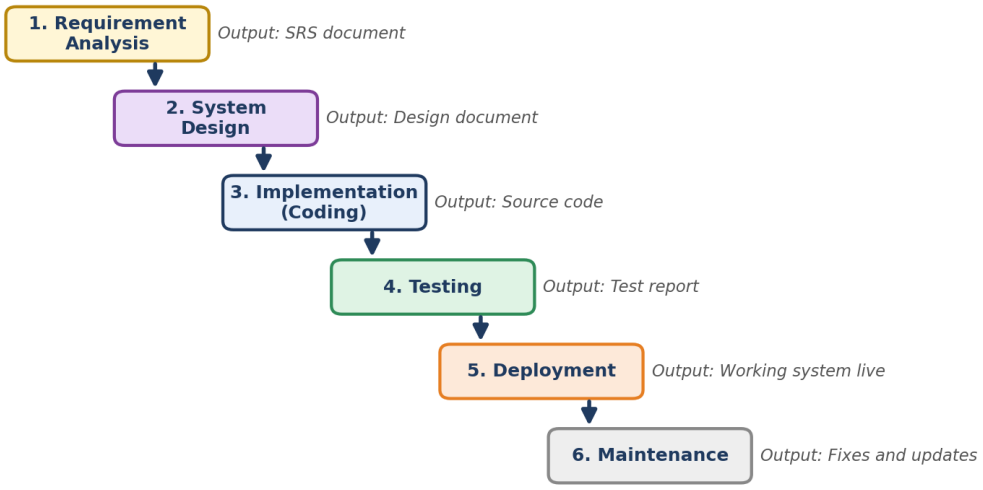


Figure: Waterfall model: phases flow downward one after another

Advantages: simple and easy to understand, easy to manage (clear milestones), good documentation, works well when requirements are fixed and well understood.

Limitations of the waterfall model

- **Rigid:** going back to an earlier phase is difficult and expensive; changes in requirements are hard to handle.
- **Late testing:** defects are found only at the testing phase, when they are costly to fix.
- **Working software appears late;** the customer sees the product only at the end, so misunderstandings are discovered late.
- **High risk and uncertainty** for long, complex or unclear projects.
- **Unsuitable** for object-oriented, evolving or maintenance-heavy projects; no early prototype.

HARD WORDS EXPLAINED - write the ones you need

SRS: Software Requirement Specification: the document listing everything the software must do

Feasibility: whether the project is practical in terms of cost, time and technology

HLD / LLD: High-Level Design (overall structure) / Low-Level Design (detailed logic of each module)

Sequential: one step after another in a fixed order

Deployment: releasing the finished software to users

Milestone: a checkpoint that marks completion of a phase

FINAL TAKEAWAY: SDLC is the ordered set of phases to build software; waterfall runs them strictly one after another, which is simple but inflexible and tests very late.

Q4. Why Software has Defects, Problem Areas of SDLC and Impact of Defects

Covers 3 Qs | 6-9 marks

This one answer covers these questions: *1) Discuss problematic areas in the software development life cycle. (Part A) | 2) Why does software have defects? Explain in detail. (Part A) | 3) What is the impact of a defect in different phases of software development? (Part B)*

BASIC IDEA - just to understand (no need to write)

Software is built by humans, and humans make mistakes - mostly because the work is **complex**, instructions are **unclear**, and time is **short**. A mistake made while writing requirements travels into design, then code. A **defect caught early costs little; the same defect caught late is very expensive**, like fixing a wrong blueprint after the house is built.

Example: A requirement says 'show discount to students' but never says how much. Developer guesses 10%, customer wanted 20%. The defect reaches production.

ANSWER - write this in the exam (draw the figure too)

Part A: Why software has defects / problematic areas in SDLC

Chain: a human **error** (mistake) creates a **defect / bug / fault** in the work product (document or code), which may cause a **failure** (wrong behaviour) when the software runs. Defects arise in every phase of the SDLC:

SDLC phase	Problem areas that cause defects
Requirements	Incomplete, ambiguous or conflicting requirements; frequent changes; poor communication with customer (the biggest source)
Design	Poor architecture, missed non-functional needs (speed, security), design not reviewed, wrong assumptions
Coding	Human mistakes, no coding standards, complex logic, inexperience, misunderstanding the design
Testing	Not enough time or coverage, weak test cases, unstable test environment, untested changes
Deployment / maintenance	Wrong configuration, patches that break working features (regression), poor documentation
Project-level	Schedule pressure, poor planning and communication, lack of training, tool or technology limitations, software complexity

Part B: Impact of a defect in different phases

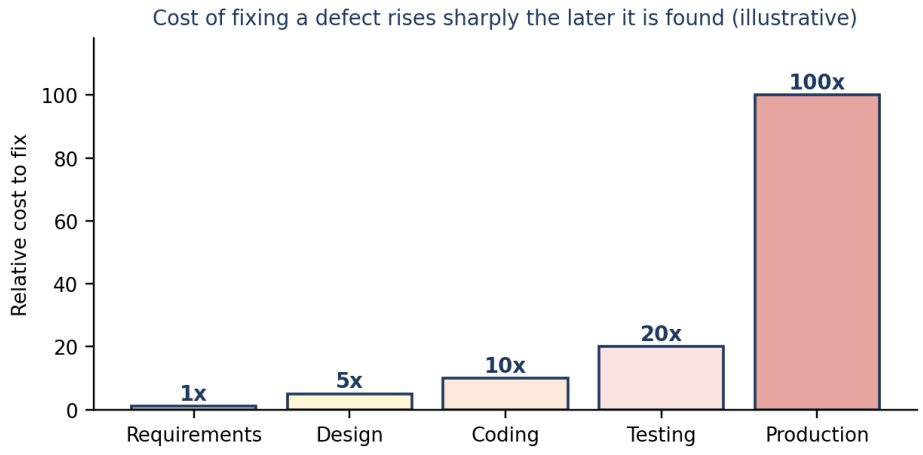


Figure: Relative cost to fix a defect by phase (illustrative values)

Phase where defect is found	Impact
Requirements	Only a document is corrected: cheapest, about 1x
Design	Redo design documents and related plans; low cost
Coding	Rewrite code, repeat reviews; moderate cost
Testing	Fix, rebuild and retest; schedule delay; higher cost
Production (after release)	Customer loss, emergency patches, reputation damage, penalties or legal action; highest cost, 100x or more

Conclusion: since cost multiplies with every phase, defects must be **prevented and detected early** through reviews and early testing.

HARD WORDS EXPLAINED - write the ones you need

Error: a human mistake made by a developer or analyst

Defect / bug / fault: a flaw in code or a document caused by an error

Failure: wrong behaviour seen when the defective software runs

Non-functional requirement: quality needs such as speed, security and usability

Regression: a change that accidentally breaks something that worked before

Ambiguous: unclear; can be understood in more than one way

FINAL TAKEAWAY: Defects come from human errors in every phase, mostly from poor requirements, and the later a defect is found the more it costs, so catch it early.

Q5. Quality Assurance vs Quality Control, and Quality Control Methods

Covers 2 Qs | 4-8 marks

This one answer covers these questions: *1) Differentiate between Quality Assurance and Quality Control. (Part A) | 2) What do you understand by quality control? Explain two methods of quality control. (Part B)*

BASIC IDEA - just to understand (no need to write)

In a **factory making biscuits**: **QA** is making sure the **recipe, machines and workers' training** are right so good biscuits come out (prevention). **QC** is **tasting a sample from the packed batch** to catch bad biscuits (detection). QA fixes the process; QC checks the product.

Example: QA: reviewing the team's coding standards and test process. QC: running the test cases on the finished software and reporting defects.

ANSWER - write this in the exam (draw the figure too)

Part A: QA vs QC

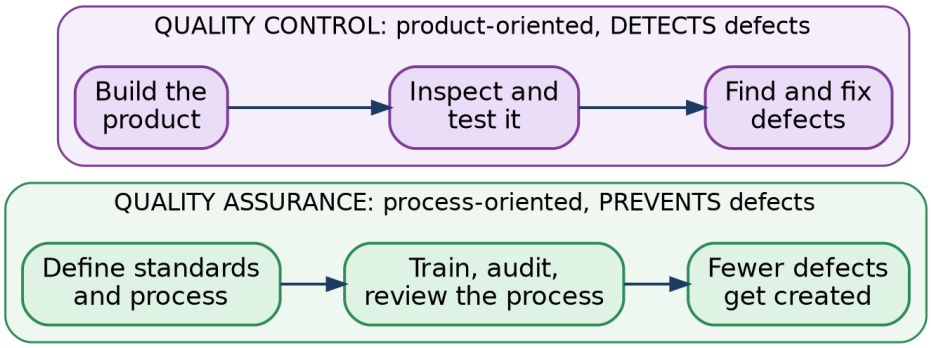


Figure: QA prevents defects in the process; QC detects defects in the product

Basis	Quality Assurance (QA)	Quality Control (QC)
Focus	Process: how the product is made	Product: the product itself
Aim	Prevent defects	Detect and correct defects
Nature	Proactive (before problems)	Reactive (after product is built)
Activities	Standards, audits, process definition, training, reviews of process	Testing, inspection, reviews of the product
When	Throughout the SDLC	After a work product / build is ready
Responsibility	Whole team and management (QA team)	Testing team / QC team
Orientation	Verification oriented	Validation oriented
Example	Process audit, checking standards are followed	System testing, finding a login bug

Part B: Quality Control and two methods

Definition: Quality Control (QC) is the set of **product-oriented activities that check the software against requirements and find defects**, so that only a product that meets quality standards is delivered.

Method 1 - Inspections and reviews (static): work products (requirements, design, code) are examined by people **without running the code**, using walkthroughs, peer reviews or formal inspections with checklists. *Example:* a team inspects the design document and finds a missing validation before any coding.

Method 2 - Testing (dynamic): the software is **executed with test cases** and the actual results are compared with expected results, at unit, integration, system and acceptance levels. *Example:* entering a wrong password in a login screen to check the error message appears.

HARD WORDS EXPLAINED - write the ones you need

Proactive / reactive: acting before a problem happens / acting after it has happened	**Static:** done without running the program
Verification: checking that we are building the product right (reviews, no execution)	**Dynamic:** done by running the program
Validation: checking that we built the right product (testing by running it)	**Walkthrough / inspection:** author explains work informally / formal review with defined roles and checklist

FINAL TAKEAWAY: QA builds quality into the process (prevention); QC checks the product (detection) mainly by inspections and testing.

Q6. ISO 9001 Standard and its Importance in Software Testing

Covers 1 Q | 6-9 marks

This one answer covers these questions: *1) Explain the ISO-9001 standard and its importance in software testing.*

BASIC IDEA - just to understand (no need to write)

ISO 9001 is like a **certificate of good management**. It does not say how to write code; it says that your company must have a **clear, documented way of working**, follow it, check it, and keep improving. An outside auditor visits and, if satisfied, gives a certificate that customers trust.

Example: A software company with ISO 9001 certification must show records of reviews, test cases, defect logs and customer feedback to the auditor.

ANSWER - write this in the exam (draw the figure too)

Definition: ISO 9001 is the international standard, published by the **International Organization for Standardization (ISO)**, that specifies the **requirements for a Quality Management System (QMS)**. An organization proves it meets them through **certification by an independent audit**. For software, the guideline **ISO/IEC 90003** helps apply it.

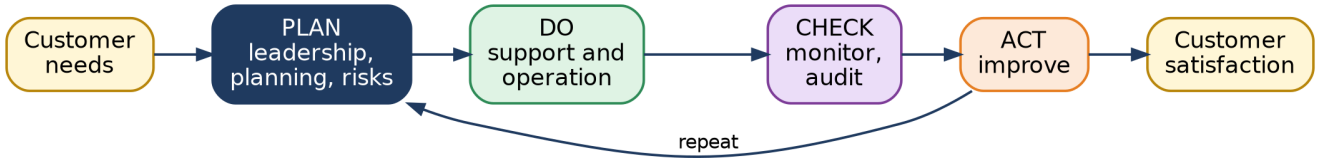


Figure: ISO 9001 uses the PDCA cycle to meet customer needs

Key ideas of ISO 9001

- **Quality management principles:** customer focus, leadership, engagement of people, process approach, improvement, evidence-based decision making, relationship management.
- **PDCA and risk-based thinking** are applied to every process.
- **Clauses:** context of the organization, leadership, planning, support, operation, performance evaluation (audits, customer satisfaction) and improvement.
- **Version:** ISO 9001:2015 is the version most syllabi teach. It has been revised as **ISO 9001:2026** (published 16 September 2026, transition till September 2029) with the same core ideas.

Importance in software testing

- Testing becomes a **documented, repeatable process** (test plans, cases, reports) instead of ad hoc.

- Gives **traceability** from requirements to tests and results, and records for audits.
- Ensures **defect tracking and corrective action**, so defects are analysed and not repeated.
- **Customer confidence and global credibility;** required by many clients and tenders.
- **Continual improvement:** test metrics and customer feedback are used to improve quality and reduce rework.

HARD WORDS EXPLAINED - write the ones you need

ISO: International Organization for Standardization, which publishes global standards

Certification: an independent body confirms in writing that you meet the standard

Audit: an official check of whether processes and records follow the standard

Process approach: managing work as connected processes with inputs and outputs

Risk-based thinking: anticipating what could go wrong and planning for it

Evidence-based: decisions made from records and data

FINAL TAKEAWAY: ISO 9001 certifies a documented, audited, continuously improving QMS; for testing it brings discipline, traceability and customer trust.

Q7. Customer Satisfaction: Why Measure it?

Covers 2 Qs | 6-9 marks

This one answer covers these questions: *1) Why do you need to measure customer satisfaction? | 2) Write a note on customer satisfaction.*

BASIC IDEA - just to understand (no need to write)

A restaurant that never asks guests how the food was will not know why customers stop coming. **Customer satisfaction** is the guest's answer to "**did the product meet what I expected?**". Measuring it tells a software company whether its quality is really good **in the user's eyes**, not only in its own tests.

Example: After release, a company sends a short survey; 80 of 100 users say 'satisfied'. CSAT = 80%. Users complain about slow loading, so the team fixes performance.

ANSWER - write this in the exam (draw the figure too)

Definition: Customer satisfaction is the degree to which a product or service **meets or exceeds customer expectations**. In software it depends on **quality (few defects), features, delivery on time, cost and support**.



Figure: Customer satisfaction improvement loop

Why we need to measure it

- To know **whether customer requirements are really met** and the product is accepted.
- To **find weak areas** (defects, performance, usability, support) and decide improvements - using data, not guesses.
- To **retain customers and build loyalty**; a satisfied customer returns and recommends; an unhappy one leaves.
- To gain a **competitive advantage** and a good reputation.
- To **reduce cost:** feedback exposes defects early, cutting rework and support effort.
- It is **required by quality standards** such as ISO 9001 and used as a quality metric.

How it is measured

Method / metric	Meaning
Surveys and questionnaires	Ask users to rate the product and service
CSAT (Customer Satisfaction Score)	% of users who answer satisfied = satisfied responses / total responses x 100
NPS (Net Promoter Score)	% Promoters - % Detractors, based on 'Would you recommend us?' (0-10)
Complaints and support tickets	Number and type of problems raised after release
Defects found by customers	Post-release defect count and severity
Interviews / focus groups	Direct discussion to understand needs

HARD WORDS EXPLAINED - write the ones you need

Expectation: what the customer believes the product should do

CSAT: Customer Satisfaction Score: percentage of satisfied customers

NPS: Net Promoter Score: shows how many would recommend the product

Loyalty: customer keeps choosing your product

Feedback: opinions and complaints given by users

Rework: doing work again because of defects

FINAL TAKEAWAY: Customer satisfaction shows how well quality is perceived by users; measuring it guides improvement, loyalty and is a requirement of ISO 9001.

Q8. Characteristics of Software + Products by Criticality

Covers 2 Qs | 6-9 marks

This one answer covers these questions: *1) Illustrate the different characteristics of software in detail. (Part A) | 2) Give types of products on the basis of criticality to the user. Explain each type with an example. (Part B)*

BASIC IDEA - just to understand (no need to write)

Part A: Software is unlike a physical machine - you cannot touch it, it does not rust, and it is built once for a particular need. A good one is judged by qualities like **working correctly, being easy to use and easy to change**. **Part B:** Different software has different stakes: a **game bug** annoys you, a **bank bug** costs money, a **pacemaker bug** can kill. The more critical, the more rigorous the testing.

Example: A calculator app (quality: functionality and usability) vs a hospital ventilator controller (life-critical).

ANSWER - write this in the exam (draw the figure too)

Part A: Characteristics of software

Characteristic	Meaning
Engineered, not manufactured	Cost lies in design and development; copying is almost free
Does not wear out	No physical ageing, but it deteriorates as changes introduce defects
Intangible	Cannot be seen or touched; judged by behaviour
Mostly custom built	Built to specific needs; limited reuse of components
Complex	Many modules and interactions make defects likely
Flexible / easy to change	Can be modified, but changes can break other parts

Quality characteristics of good software (ISO 9126; newer ISO 25010 adds Security and Compatibility):

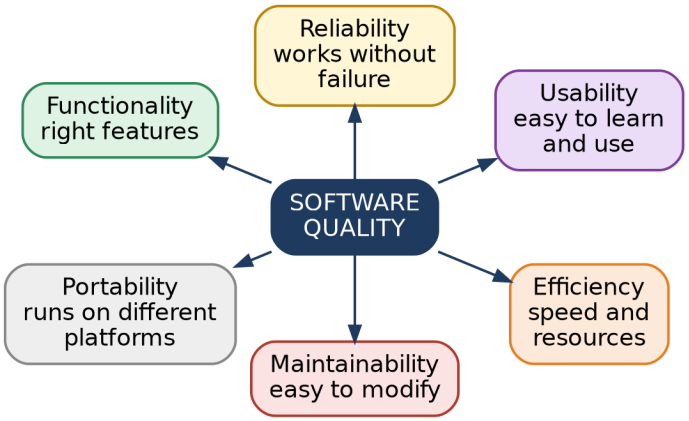


Figure: Quality characteristics of software

Part B: Types of products on the basis of criticality

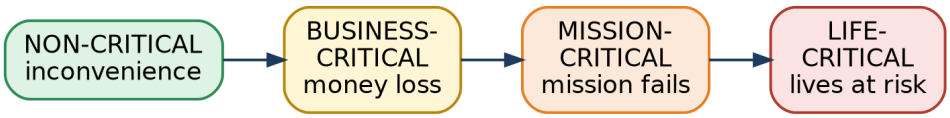


Figure: From non-critical to life-critical: testing rigor increases

Type	Meaning (effect of failure)	Example
Life-critical (safety-critical)	Failure can cause loss of life or serious injury	Pacemaker, flight control, ventilator, railway signalling
Mission-critical	Failure makes an important mission or operation fail	Space mission control, air traffic control, defence systems
Business-critical	Failure causes major financial loss or business stoppage	Banking, ATM, stock trading, e-commerce, airline reservation
Non-critical	Failure only causes inconvenience	Games, media players, personal utilities

HARD WORDS EXPLAINED - write the ones you need

Intangible: has no physical form
Reliability: ability to work without failure for a given time
Usability: how easy the software is to learn and use
Maintainability: how easily it can be corrected or modified
Portability: ability to run in different environments
Safety-critical: failure can harm human life

FINAL TAKEAWAY: Software is an intangible, engineered product judged by quality characteristics, and the more critical its use, the more rigorous its testing must be.

Q9. Selenium IDE

Covers 1 Q | 6-9 marks

This one answer covers these questions: *1) Illustrate Selenium's IDE and explain it in detail.*

BASIC IDEA - just to understand (no need to write)

Normally, testing a website means clicking through it again and again. **Selenium IDE works like a tape recorder**: you press record, use the website once, and it **records every click and typing**. Press play and it repeats the exact steps automatically - **no programming needed**.

Example: Record a login test: open site, type username, type password, click Login, check the welcome message. Replay it after every new build.

ANSWER - write this in the exam (draw the figure too)

Definition: Selenium IDE (Integrated Development Environment) is a free, open-source **browser extension** (Chrome, Firefox, Edge) for **recording, editing and replaying** automated tests of web applications. It is the simplest tool of the **Selenium suite**.

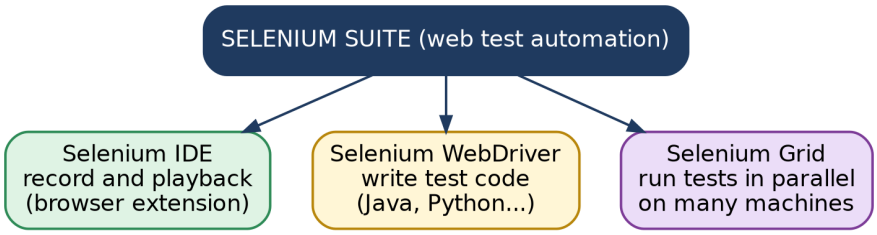


Figure: Selenium suite and the role of IDE

Working (record and playback)

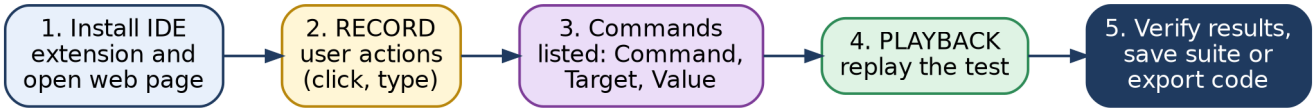


Figure: Steps in using Selenium IDE

Features

- **Record and playback** of browser actions without coding; the steps appear as commands (**Command, Target, Value**).
- **Locators** (id, name, CSS, XPath) to find page elements; **assertions / verify** commands to check results.
- **Edit and debug:** step-by-step execution, breakpoints, variables, control flow (if, loops).
- **Test suites** to group tests; **export** to WebDriver code (Java, Python, C#, JavaScript); run from command line with the **selenium-side-runner**.

Example test (login)

Command	Target	Value
open	/login	
type	id=username	student01
type	id=password	Pass@123
click	id=loginBtn	
assert title		Dashboard

Advantages: easy for beginners, quick to create tests, free, good for prototyping and regression checks. **Limitations:** only for web applications in browsers, weak for complex logic and large data-driven frameworks, no built-in parallel execution (use Grid / WebDriver), recorded locators can become fragile.

HARD WORDS EXPLAINED - write the ones you need

Browser extension: a small add-on installed in a web browser

Record and playback: capture user actions, then replay them automatically

Locator: the address of an element on a page (id, CSS, XPath)

Selenese: Selenium's command language (open, type, click, assert)

Assertion: a check that the actual result equals the expected result

Regression testing: re-running tests to make sure new changes did not break old features

FINAL TAKEAWAY: Selenium IDE is a record-and-playback browser extension that lets you create and replay automated web tests without programming.

STQA - UNIT 5: Test Automation, Selenium & Performance Testing

END-SEM EXAM NOTES

How to use: Every answer is for **6 to 9 marks** (write fewer parts for fewer marks). **Yellow box** = basic idea, only to understand (do not write). Everything after **ANSWER** = write in the exam (draw the figure too). **Blue box** = hard words explained simply - write the ones you need. Where an answer has **Part A / Part B**, each part is complete by itself: write only the part asked, or both if the question joins them. **Contents** (bracket = how many past question wordings one answer covers; freq = how often asked): A1 Automation Testing + Manual vs Automation (3; freq 2+1) | A2 Automated Testing Process (2; freq 4) | A3 Benefits + Choosing a Tool (3; freq 3+1) | A4 RPA (1; freq 3) | A5 Selenium Suite, IDE, WebDriver, RC, Grid (5; freq 2+2+2+1+1) | A6 Performance Testing (1; freq 1).

Find the right answer quickly:

If the question says...	Write
What is automation testing / explain with example / in brief	A1 Part A
Manual testing vs automation testing	A1 Part B
Automated testing process / construct processes / with neat diagram	A2
Benefits of automation testing (identify / list and explain)	A3 Part A
How to choose an automation tool	A3 Part B
Robotic Process Automation (RPA)	A4
Selenium tool suite / illustrate the suite	A5 Part A
Selenium IDE	A5 Part B
Selenium WebDriver	A5 Part C
Selenium RC	A5 Part D
Selenium Grid	A5 Part E
Performance testing and its uses	A6

A1. Automation Testing + Manual vs Automation

Covers 3 Qs | 6-9 marks

This one answer covers these questions: 1) What is automation testing in software testing? Explain in brief. (Part A) | 2) What is automation testing? Explain it with an example. (Part A) | 3) Differentiate between manual testing and automation testing. (Part B)

BASIC IDEA - just to understand (no need to write)

Imagine checking a login page with 50 different username-password pairs. Typing them one by one is slow, boring and you may make mistakes. Now imagine a small program that types all 50 pairs, clicks Login and checks the message shown - in a minute, exactly the same way every time. That is automation testing: **let a tool do the repeated checking so humans can do the thinking.**

ANSWER - write this in the exam (draw the figure too)

Part A: Automation Testing

Definition: Automation testing is a software testing technique in which special **tools and test scripts** execute test cases automatically, compare the **actual result** with the **expected result**, and report pass or fail with very little human effort.

How it works:

1. Testers write test cases (steps + expected result).
2. Test cases are turned into **test scripts** (small programs) using an automation tool.
3. The tool runs the scripts on the **application under test (AUT)**.
4. Actual result is compared with expected result automatically.
5. A report of passed and failed tests is generated.

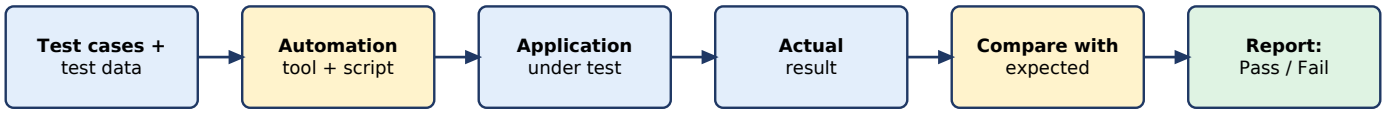


Fig: How an automated test runs

Where to use: regression testing, repetitive tests, tests with large data, cross-browser or multi-platform tests, load tests, and quick checks after every new build. **Where not to use:** one-time tests, exploratory and usability testing, and screens that change very often.

Example

Online shopping site. Test case: *"Login with a valid user must show the Welcome page."* Script steps: open browser → go to site URL → type username → type password → click Login → check that the page contains "Welcome" → close browser. The same script runs on Chrome and Firefox every night after developers change code. If "Welcome" is missing, the test is marked **Fail** and the tester is informed. Tools used: Selenium, UFT, TestComplete.

Advantages: fast, accurate, reusable, saves cost in the long run. **Limitations:** high initial cost, needs programming skill, scripts need maintenance, cannot judge look and feel.

Part B: Manual Testing vs Automation Testing

Point	Manual testing	Automation testing
Basic idea	Tester executes test cases step by step by hand	Tools and scripts execute test cases
Speed	Slow and time consuming	Very fast; can run overnight
Accuracy	Human errors are possible	Reliable; same steps every time
Initial cost	Low	High (tool, setup, writing scripts)
Long-term cost	High for repeated tests	Low because scripts are reused
Skills needed	Domain and testing knowledge	Programming and tool knowledge
Reusability	Effort cannot be reused	Scripts reused many times
Best suited for	Exploratory, usability, ad-hoc, one-time tests	Regression, load, repetitive, large-data tests
Human judgement	Can judge look, feel and ease of use	Cannot judge appearance or user-friendliness
Parallel testing	Not practical	Many browsers or machines at once
Reports	Prepared manually	Generated automatically

HARD WORDS EXPLAINED SIMPLY

Test script - a set of instructions (a small program) telling the tool what to do on the application.
Test case - steps to perform plus the expected result.
AUT - Application Under Test - the software being tested.
Regression testing - re-testing after a change to make sure old features still work.
Exploratory testing - testing by freely exploring the application without a fixed script.

FINAL TAKEAWAY

Automation testing uses tools and scripts to run tests quickly and repeatedly, best for regression and repetitive work; manual testing is still needed where human judgement is required.

A2. Automated Testing Process

Covers 2 Qs | 6-9 marks

This one answer covers these questions: 1) Construct the different automated testing processes. (Complete answer) | 2) With a neat diagram, explain the automated testing process. (Complete answer)

BASIC IDEA - just to understand (no need to write)

Automating tests is like setting up a small factory. First decide what to make (what to automate), choose the machines (tools), build the production line (framework and scripts), run it, check the output, and keep repairing the line whenever the product design changes. The automated testing process is simply these stages in order.

ANSWER - write this in the exam (draw the figure too)

Automated Testing Process

The automated testing process is a series of stages that starts with deciding **what** to automate and ends with **maintaining** the scripts, then repeats for the next release.

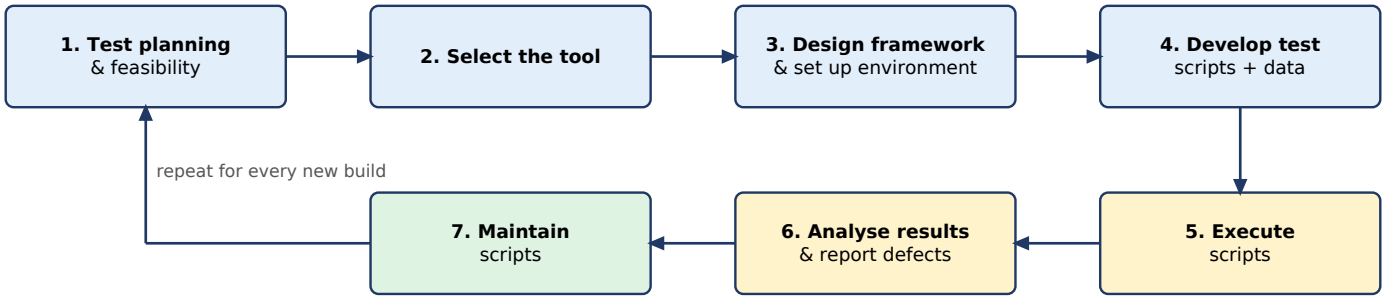


Fig: Automated testing process (7 stages with feedback loop)

Stages explained

1. **Test planning and feasibility:** decide what to automate and what not (choose stable, repetitive, high-risk, data-heavy tests). Estimate effort, time and **ROI**. *Output: automation plan.*
2. **Select the tool:** choose by application type, cost, team skills and support (see A3 Part B).
3. **Design framework and set up environment:** a **framework** is the structure/guidelines for organising scripts, test data and reports. Prepare machines, browsers and test data.
4. **Develop test scripts:** write or record scripts, connect test data, add **assertions** (checks of actual vs expected result) and review the scripts.
5. **Execute scripts:** run manually or on a schedule (for example nightly after a new build); can run in parallel.
6. **Analyse results and report:** study pass/fail logs, separate real defects from script problems, log defects in a tracker and share the report.
7. **Maintain scripts:** update scripts when the application changes, remove outdated ones, improve and re-run.

Supplementary: different approaches used to build the scripts (my addition - write if the question says “different processes”)

Approach	Meaning	Note
Record and playback	Tool records user actions and replays them	Quick but fragile when screens change
Modular	Application split into small modules, each with its own script	Easy to maintain and reuse
Data-driven	Same script runs with many inputs read from Excel/CSV	Good for many data combinations
Keyword-driven	Actions written as keywords (Login, Click) in a table	Non-programmers can build tests
Hybrid	Mix of the above approaches	Most real frameworks

STQA-5

HARD WORDS EXPLAINED SIMPLY
Framework - ready structure of folders, rules and reusable code that keeps scripts organised.
Assertion - a check inside a script that compares actual and expected result.
ROI - Return On Investment - benefit gained compared with money and effort spent.
Build - a new version of the software prepared by developers.
Test data - the input values (usernames, amounts etc.) used by the tests.

FINAL TAKEAWAY
Plan → choose tool → build framework → write scripts → execute → analyse → maintain; the cycle repeats with every release.

A3. Benefits of Automation + Choosing a Tool

Covers 3 Qs | 6-9 marks

This one answer covers these questions: 1) Identify the different benefits of automation testing. (Part A) | 2) List and explain the benefits of automation testing. (Part A) | 3) How to choose an automation tool? Explain briefly. (Part B)

BASIC IDEA - just to understand (no need to write)
A calculator does not make you smarter; it makes repeated sums fast and error-free so you can think about harder problems. In the same way, automation takes over the boring repeated checks, giving faster results and leaving time for creative testing.

ANSWER - write this in the exam (draw the figure too)

Part A: Benefits of Automation Testing

Benefit	Explanation
1. Saves time	Scripts run much faster than humans and can run unattended, even overnight.
2. Reusability	Write once and run on every build, release and environment.
3. Accuracy and reliability	No tiredness or typing mistakes; the same steps run exactly each time.
4. Saves cost in the long run	Initial cost is high, but repeated test cycles need far less effort.
5. Better test coverage	Thousands of data combinations and more test cases run in the same time; even load tests with many virtual users become possible.
6. Ideal for regression testing	Quickly confirms that old features still work after every change.
7. Parallel and cross-browser testing	Same test runs on many browsers, devices and operating systems at once (for example with Selenium Grid).
8. Early defect detection	With **CI** tests run on every code change, so bugs are found sooner and are cheaper to fix.
9. Automatic reports and logs	Detailed pass/fail results and screenshots serve as evidence.
10. Frees testers	Testers can focus on exploratory and usability testing; this also improves team morale.

Keep in mind (limitations): high initial cost, scripts need maintenance, cannot judge look and feel, not suitable for one-time tests.

Part B: How to Choose an Automation Tool

A wrong tool wastes money and effort, so the choice must be based on clear criteria.

Criteria

- **Application type and technology:** web, mobile, desktop or API - the tool must support it.
- **Cost and licence:** open-source (free) or commercial (paid, with vendor support).
- **Team skills:** programming languages supported (Java, Python...) or codeless options available.
- **Ease of learning and use,** and available training and documentation.
- **Browser, OS and device support** needed for the project.
- **Integration** with CI tools (Jenkins), defect trackers (Jira) and version control.
- **Reporting and debugging features;** reliable identification of UI elements, including dynamic content.
- **Community and vendor support, updates, scalability** (parallel runs) and maintenance effort.

Steps to decide

1. List requirements.
2. Shortlist tools.
3. Try a small **proof of concept (PoC)** on a real part of the application.
4. Compare using a score card.
5. Decide and train the team.

Examples: Selenium (free, web applications, many languages), Appium (free, mobile apps), UFT (commercial, web and desktop).

HARD WORDS EXPLAINED SIMPLY
Open-source - free tool whose code is public and community-supported.
Commercial tool - paid tool with licence and vendor support.
CI (Continuous Integration) - code is merged often and tests run automatically on every change.
Proof of concept - a small trial to check that a tool really works for your project.
Cross-browser testing - checking the app works in different browsers.

FINAL TAKEAWAY
Automation gives speed, accuracy, reuse and wide coverage; pick the tool that fits your application, team skills, budget and integrations after a small trial.

A4. Robotic Process Automation (RPA)

Covers 1 Q | 6-9 marks

This one answer covers these questions: 1) Explain Robotic Process Automation (RPA) in detail. (Complete answer)

BASIC IDEA - just to understand (no need to write)
An office clerk opens an email, copies the invoice amount, pastes it into accounting software and clicks Save - the same clicks, same rules, hundreds of times a day. RPA gives that clerk a **software robot** that does exactly those clicks and typing on the screen, so people can do thinking work.

ANSWER - write this in the exam (draw the figure too)

Robotic Process Automation (RPA)

Definition: RPA is a technology that uses **software robots (bots)** to imitate human actions on a computer - logging in, clicking, typing, copying and pasting data, reading emails and files, filling forms - to automate **repetitive, rule-based** business processes. The robots are software, not physical machines.

Key characteristics

- Works through the **user interface** like a human, so existing systems need not be changed.
- Follows fixed rules; works 24x7 without breaks.
- Low-code: workflows can be built by recording or drag-and-drop.

How it works

1. Identify a suitable repetitive process.
2. Design the workflow in the RPA designer (record steps or drag-and-drop).
3. Build and test the bot.
4. Deploy the bot to run.
5. Monitor through the orchestrator; humans handle exceptions the bot cannot.

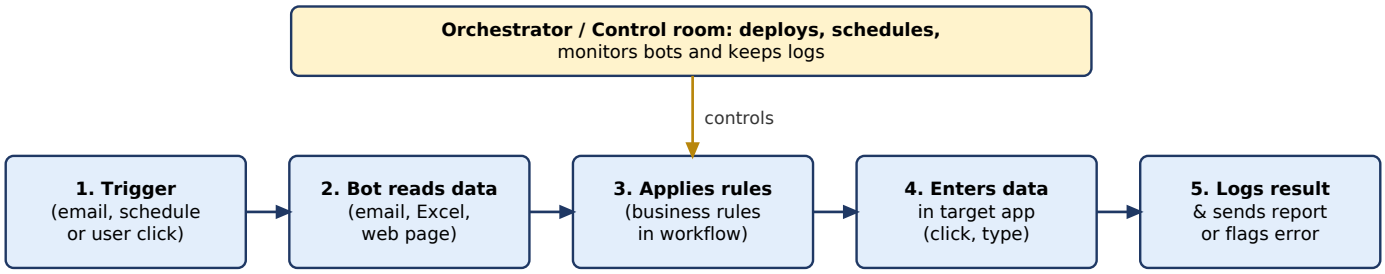


Fig: A software robot (bot) performing a rule-based task

Component	Role
Designer / Studio	Used to build and edit automation workflows
Bot / Robot	Executes the workflow on a computer or server
Orchestrator / Control room	Deploys, schedules and monitors bots; stores logs

Type of bot	Meaning	Example
Attended	Runs on the user's desktop beside the human; started by the user	Helps a call-centre agent fill forms
Unattended	Runs on a server automatically on schedule; no human needed	Night-time invoice processing
Hybrid	Mix of both	Bot prepares data, human approves

Example: invoice processing

Bot opens the inbox → downloads the invoice PDF → reads invoice number and amount → enters them in the accounting software → saves and sends a confirmation email → writes a log. Unreadable invoices are sent to a human as exceptions.

Benefits: fast, accurate, 24x7, saves cost, scalable, gives an audit trail, frees employees. **Limitations:** works only on structured, rule-based tasks; breaks if screen layout changes; setup and maintenance effort; cannot use judgement.

Applications: banking (KYC, account opening), insurance claims, HR onboarding and payroll, finance (invoices, reconciliation), healthcare billing, IT user-account creation. **Tools:** UiPath, Automation Anywhere, Blue Prism.

RPA vs test automation: both imitate user actions on the screen, but RPA does real business work, while test automation checks that software works correctly.

HARD WORDS EXPLAINED SIMPLY

Rule-based - task that follows clear if-then rules, with no personal judgement.
Structured data - data in a fixed format such as Excel sheets or forms.
Orchestrator - the control centre that manages many bots.
KYC - Know Your Customer - verifying a customer's identity at a bank.
Low-code - building automation with little or no programming.

FINAL TAKEAWAY

RPA uses software bots to copy human screen actions for repetitive, rule-based business tasks - faster, accurate and available 24x7.

A5. Selenium Tool Suite, IDE, WebDriver, RC, Grid

Covers 5 Qs | each Part 6 marks

This one answer covers these questions: 1) Illustrate the Selenium tool suite in detail. (Part A) | 2) Explain Selenium IDE in detail. (Part B) | 3) How would you explain Selenium WebDriver? (Part C) | 4) What is Selenium RC? Explain in detail. (Part D) | 5) What do you understand by Selenium Grid? (Part E)

BASIC IDEA - just to understand (no need to write)

Suppose you must test a website on Chrome, Firefox and Edge, on Windows and Linux. Doing it by hand is slow. Selenium is a free toolbox for this: one tool records your actions (**IDE**), one controls the browser through code (**WebDriver**), one runs tests on many machines together (**Grid**), and an old one (**RC**) was replaced by WebDriver. Together they are the Selenium Suite. **If only the suite is asked, write Part A; it already has a short note on each component.**

ANSWER - write this in the exam (draw the figure too)

Part A: Selenium Tool Suite

Selenium is a free, open-source suite for automating **web application** testing across browsers and operating systems. Started by Jason Huggins in 2004, it supports languages such as Java, Python, C#, Ruby and JavaScript.

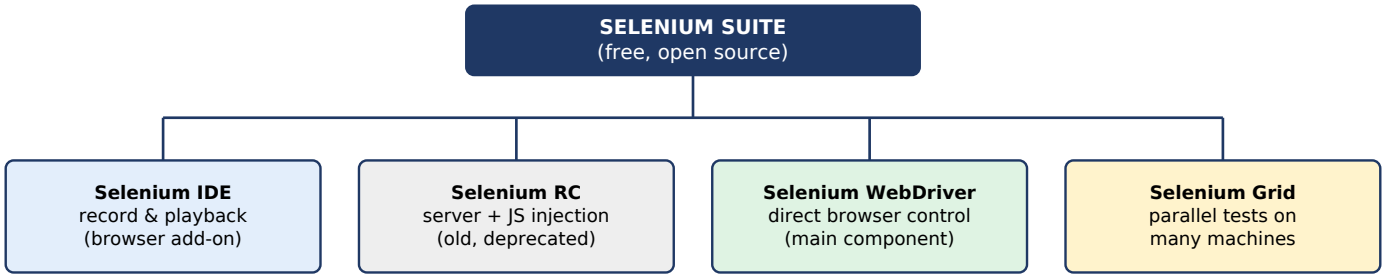


Fig: Components of the Selenium tool suite

Component	What it is	Coding needed?	Status
Selenium IDE	Browser extension that records and plays back actions	No	Active
Selenium RC	Java server that injects JavaScript into the browser	Yes	Deprecated; replaced by WebDriver
Selenium WebDriver	API that controls the browser directly	Yes	Main and most used
Selenium Grid	Runs tests in parallel on many machines	Yes (setup)	Active; redesigned in Selenium 4

Versions in short: Selenium 1 = RC; Selenium 2 = WebDriver added; Selenium 3 = RC removed; Selenium 4 = W3C WebDriver standard, redesigned Grid, relative locators, updated IDE.

Advantages: free; many languages, browsers and operating systems; large community; works with TestNG, JUnit, Maven and Jenkins. **Limitations:** web applications only (Appium is used for mobile); no built-in reporting; needs programming skill; no official vendor support; cannot handle CAPTCHA.

Part B: Selenium IDE

Definition: Selenium IDE (Integrated Development Environment) is a browser extension for Chrome, Firefox and Edge that **records** user actions and **plays them back** as an automated test. No programming is needed.

Features

- Record and playback of browser actions.
- Commands are shown in a table: **Command | Target | Value** (the command language is called **Selenese**: actions like open, click, type; checks like assert and verify).
- Automatically picks locators, and offers alternative locators.
- Breakpoints and step-by-step debugging; tests can be grouped into suites.
- Control-flow commands (if, while, times) in the new IDE; tests can be run from the command line.
- Test can be **exported** to WebDriver code (Java, Python, C#, JavaScript).

Steps of use

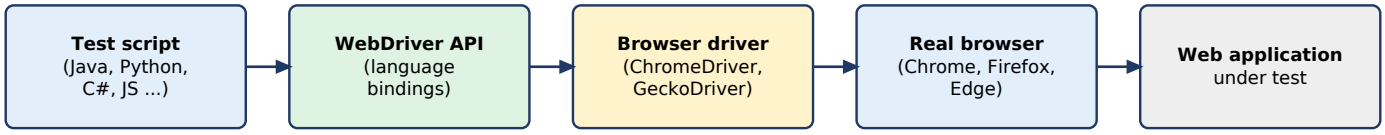
1. Install the extension; create a project and enter the base URL.
2. Click Record and perform the actions on the website.
3. Stop recording and play the test back.
4. Add assert commands to check results.
5. Save (.side file) or export as code.

Example: record login: open site → type username → type password → click Login → assert text “Welcome”. Replay it on any build.

Advantages: easy for beginners, fast prototyping, free. **Limitations:** not suitable for complex logic or large data-driven tests; less control and harder to maintain; only web and only browsers that support the extension.

Part C: Selenium WebDriver

Definition: Selenium WebDriver is the core of Selenium - an open-source **API** (set of commands) that lets a program written in Java, Python, C# etc. **control a real browser directly**, like a user: open a URL, click, type, read text, switch windows and alerts.



Commands go left to right (W3C WebDriver protocol over HTTP); results come back right to left.

Fig: Selenium WebDriver architecture

Key points

- **Direct communication** with the browser through a **browser driver** (ChromeDriver, GeckoDriver for Firefox, EdgeDriver); no extra server in between.
- Selenium 4 uses the **W3C WebDriver protocol** - an official standard - between script and driver.
- Finds elements using **locators**: id, name, class name, tag name, link text, partial link text, CSS selector, XPath.
- Handles alerts, frames, multiple windows, drop-downs, mouse and keyboard actions, screenshots and **waits** for slow-loading pages.
- Works with TestNG/JUnit for reports, and with Grid through RemoteWebDriver.

Basic flow of a WebDriver test

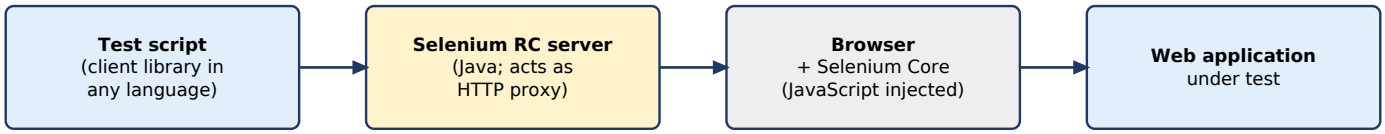
Launch browser → open URL → find element using locator → perform action → verify with assertion → close browser.

Why better than RC: faster, simpler, no extra server, closer to real user behaviour. **Limitations:** programming needed; web only; driver version must match browser version; no built-in reporting.

Part D: Selenium RC (Remote Control)

Definition: Selenium RC was the first Selenium tool for browser automation: a **Java server** that received commands from test scripts (in any supported language) and made the browser carry them out.

Why it was needed: early Selenium Core was JavaScript running inside the browser. Because of the browser's **same-origin policy**, JavaScript from one site could not control a page of another site. RC acted as an **HTTP proxy** so that the browser believed Selenium Core and the website came from the same server.



Script sends HTTP commands to the server; the server injects JavaScript into the browser, which performs them.

Fig: Selenium RC working

Working

1. Test script sends a command to the RC server as an HTTP request.
2. Server launches the browser and **injects Selenium Core** (JavaScript) into it.
3. Core performs the action on the page (click, type, etc.).
4. Result goes back to the server and then to the script, which continues.

Components: client libraries, RC server, Selenium Core. **Advantages:** supported many languages and browsers and allowed complex tests unlike IDE. **Limitations:** slower (extra server and JavaScript injection), complex setup, JavaScript sandbox problems (for example file upload, pop-ups), cluttered API. **Status:** deprecated; replaced by WebDriver, so use WebDriver today.

Part E: Selenium Grid

Definition: Selenium Grid lets WebDriver tests run on **many machines, browsers and operating systems at the same time (in parallel)**, managed from one central point.

Need: one machine cannot have every browser and OS combination, and running 100 tests one after another takes hours.

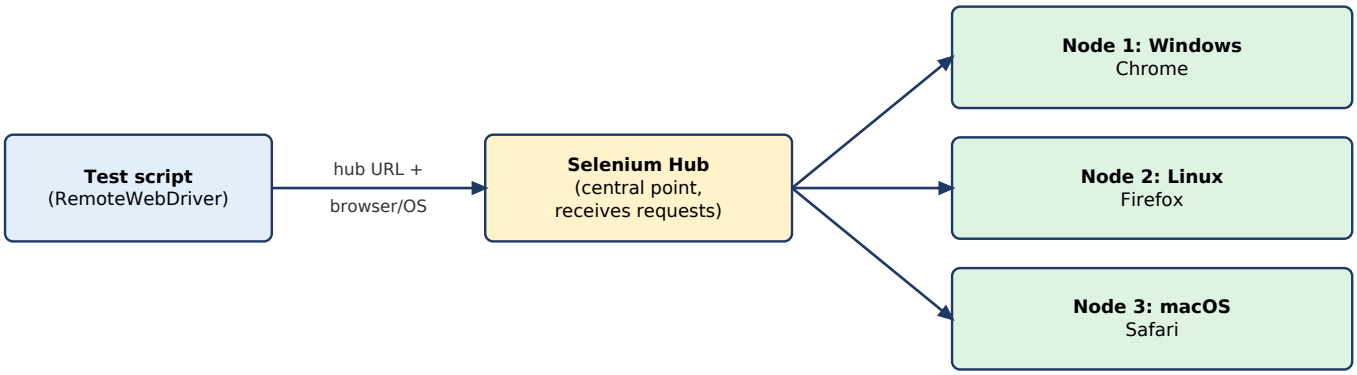


Fig: Selenium Grid - one hub distributes tests to many nodes in parallel

Architecture

- **Hub:** the central server that receives test requests and sends each to a suitable node.
- **Nodes:** machines with browsers; they register with the hub and run the tests.
- The script uses **RemoteWebDriver** with the hub URL and the required browser/OS (capabilities).
- Selenium 4 Grid can run as **Standalone**, **Hub and Node** or **Fully Distributed**, with Router, Distributor, Session Map, Session Queue, Node and Event Bus.

Benefits: parallel execution saves time, cross-browser and cross-OS coverage, scalable (add more nodes), reuses existing machines. **Limitations:** setup and maintenance of machines, network delays, tests must be independent of each other. **Illustration:** if 90 tests take 90 minutes one by one, three nodes can finish in about 30 minutes.

STQA-5

HARD WORDS EXPLAINED SIMPLY
Locator - the way to find an element on a page: id, name, class, XPath, CSS selector, link text.
Element - any item on a web page such as a button, text box or link.
Selenese - Selenium's own command language used in IDE and RC.
Same-origin policy - browser rule that stops JavaScript of one website from controlling another site's pages.
Browser driver - small program that lets Selenium talk to a particular browser.
Hub / Node - hub = central manager; node = machine that runs tests.
Parallel execution - running many tests at the same time.
Wait - an instruction to pause until the page or element is ready.

FINAL TAKEAWAY
Selenium = IDE (record), RC (old server), WebDriver (direct browser control, the main one) and Grid (parallel runs on many machines).

A6. Performance Testing

Covers 1 Q | 6-9 marks

This one answer covers these questions: 1) What is performance testing? Explain its uses as well. (Complete answer)

BASIC IDEA - just to understand (no need to write)
A bridge may look perfect, yet engineers test how many vehicles it can carry and for how long. Similarly an app may work correctly for one user but become slow or crash when thousands use it together. Performance testing checks **how fast and stable** the software is under load.

ANSWER - write this in the exam (draw the figure too)

Performance Testing

Definition: Performance testing is a type of **non-functional testing** that checks the **speed, responsiveness, stability, scalability and resource usage** of an application under expected and extreme workloads. It checks *how well* the software works, not *what* it does.

Uses (objectives)

- Measures **response time** and speed for users.
- Finds **bottlenecks** (the slowest part) in the code, database, server or network.
- Checks **stability** - the system should not crash under heavy use.
- Checks **scalability** - can it handle growth in users or data?
- Checks resource usage (CPU, memory, disk, network) to help capacity planning.
- Avoids failures on launch or sale days and protects the company's reputation.
- Confirms that agreed performance targets are met.

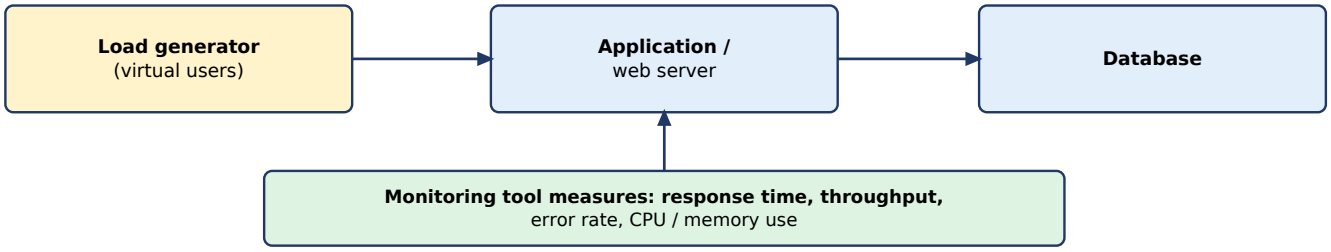


Fig: Performance testing set-up

Types

Type	Meaning
Load testing	Checks behaviour under the expected number of users
Stress testing	Pushes the system beyond its limit to see where and how it fails and recovers
Spike testing	Sudden big rise or fall in users
Endurance (soak) testing	Normal load for a long time to find memory leaks and slow-downs
Volume testing	Large amount of data in the database
Scalability testing	Checks if performance holds as users or resources are increased

Process

Set goals and metrics → prepare test environment similar to live → design scenarios and scripts → execute with virtual users → monitor and collect metrics → analyse and find bottlenecks → tune and re-test.

STQA-5

Metrics: response time, throughput (requests per second), number of concurrent users, error rate, CPU and memory use. **Tools:** Apache JMeter, LoadRunner, Gatling, k6.

Example: a shopping website normally has 500 users but expects 10,000 on a festival sale day. A load test creates 10,000 **virtual users** who search, add to cart and pay, while the tester watches response time and server load. If the pages take too long, the slow database query is found and fixed before the sale. Such tests cannot be done manually, so automation tools are used.

HARD WORDS EXPLAINED SIMPLY

Non-functional testing - testing qualities such as speed and security, not features.
Virtual user - a simulated user created by the tool to generate load.
Bottleneck - the slowest part that limits the whole system.
Throughput - amount of work done per second, for example requests per second.
Response time - time between a request and its response.

FINAL TAKEAWAY

Performance testing checks speed, stability and scalability under load, so bottlenecks are found and fixed before real users face them.

STQA - UNIT 6: Six Sigma, SQA, Quality Tools & TQM

END-SEM EXAM NOTES

How to use: Every answer is for **6 to 9 marks** (write fewer parts for fewer marks). **Yellow box** = basic idea, only to understand (do not write). Everything after **ANSWER** = write in the exam (draw the figure too). **Blue box** = hard words explained simply - write the ones you need. Where an answer has **Part A / B / C**, each part is complete by itself: write only the part asked, or several if the question joins them. **Contents** (bracket = how many past questions one answer covers): Q1 Six Sigma: characteristics and strategy (2) | Q2 Achieving software quality: parameters, facts, activities (3) | Q3 Seven QC tools, flowchart / run chart / control chart, fishbone vs histogram (6) | Q4 SQA plan, tasks-goals-metrics, defect removal effectiveness (3) | Q5 Total Quality Management (1) | Q6 Software maintenance and maintaining SQA (2).

Find the right answer quickly:

If the question says...	Write
Six Sigma characteristics / Six Sigma strategy of SQA	Q1 (one answer serves both)
Parameters / facts / activities for achieving software quality	Q2 (all three are inside)
Ishikawa's 7 basic tools / basic tools for quality control / histogram, flowchart and control chart in brief	Q3 Part A
Compare run chart and control chart / clarify flowchart, run chart, control chart	Q3 Part B
Compare Ishikawa's diagram (fishbone) and histogram	Q3 Part C
SQA plan / tasks, goals and metrics in SQA / defect removal effectiveness	Q4 Part A / Part B / Part C
Total Quality Management	Q5
Software maintenance / how to maintain SQA	Q6 Part A / Part B

Q1. Six Sigma: Characteristics and Strategy in SQA

Covers 2 Qs | 6-9 marks

> ***This one answer covers these questions:*** *1) Explain the Six Sigma characteristics in detail. | 2) Write a note on the Six Sigma strategy of software quality assurance.*

BASIC IDEA - just to understand (no need to write)

Sigma (σ) measures how much a process wanders away from perfect. **Six Sigma is a method to make a process so consistent that it produces almost no defects** - only **3.4 defects in every million chances**. Imagine a courier that delivers on time 99.99966% of the time. Six Sigma finds *why* things go wrong using data and fixes the cause, step by step.

Example: A bank app processes 1,000,000 transactions. At Six Sigma quality only about 3 fail. A 3-sigma process would fail about 66,807 of them.

ANSWER - write this in the exam (draw the figure too)

Definition: Six Sigma is a **data-driven, disciplined quality methodology** (started at Motorola in 1986, made famous by General Electric) that improves a process by **removing the causes of defects and reducing variation**, with the target of **3.4 defects per million opportunities (DPMO)**.

Sigma level	Defects per million (DPMO)	Yield (defect-free)
1 sigma	691,462	30.9%
2 sigma	308,538	69.1%
3 sigma	66,807	93.3%
4 sigma	6,210	99.38%
5 sigma	233	99.977%
6 sigma	**3.4**	**99.99966%**

Characteristics of Six Sigma

Characteristic	Meaning
Customer focus	Quality is defined by **CTQ** (Critical To Quality) needs of the customer
Data-driven decisions	Facts and statistics, not opinions, are used to find and fix problems
Process focus	Improve the process that produces the product, not just inspect the product
Defect and variation reduction	Goal is 3.4 DPMO; reduce variation so results are predictable
Proactive (prevention)	Prevent defects early instead of finding them late
Structured method	Projects follow **DMAIC** (or DMADV) with clear steps
Team and trained roles	Champions, Master Black Belts, Black Belts, Green Belts, Yellow Belts lead and support projects
Management commitment and continuous improvement	Top-level support; every project must show measurable benefit

Six Sigma strategy for software quality assurance (DMAIC)

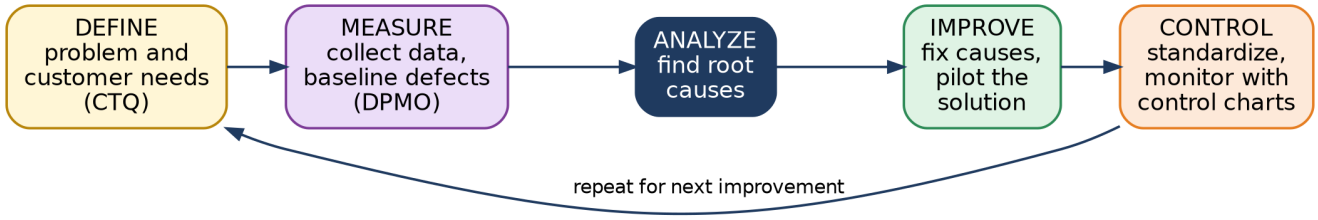


Figure: DMAIC cycle of Six Sigma

Phase	Activity	Software example (too many post-release defects)
Define	State the problem, goal and customer CTQ; form the team	Customers report too many bugs after release
Measure	Collect data, set the baseline (defects/KLOC, DPMO)	Current: 8 defects per KLOC
Analyze	Find root causes with fishbone, Pareto, scatter diagrams	60% of defects come from unclear requirements
Improve	Implement and pilot solutions	Add requirement reviews, checklists, more unit tests
Control	Standardize and monitor with control charts	Track defect density every release; keep it below 2/KLOC

*For designing a new process or product the variant **DMADV** is used: Define, Measure, Analyze, **Design, Verify**.*

HARD WORDS EXPLAINED - write the ones you need

Sigma (σ): a statistics symbol for spread (standard deviation); more sigma = more consistent

DPMO: Defects Per Million Opportunities: defects divided by chances for a defect, times one million

Variation: difference in results each time the same process is run

CTQ: Critical To Quality: what the customer values most

Root cause: the real, underlying reason a defect occurs

Black Belt: trained expert who leads Six Sigma projects (Green Belt = part-time practitioner)

KLOC: thousand lines of code

FINAL TAKEAWAY: Six Sigma uses data and the DMAIC cycle to cut variation and defects to 3.4 per million, making software quality predictable and customer-focused.

Q2. Achieving Software Quality: Parameters, Facts and Activities

Covers 3 Qs | 6-9 marks

This one answer covers these questions: *1) What parameters are required for achieving good software quality? | 2) What are the facts for achieving software quality? Explain in detail. | 3) Explain the activities to achieve high software quality in detail.*

BASIC IDEA - just to understand (no need to write)

Quality is like a good cake: you need **the right ingredients** (skilled people, clear requirements, good tools), **a proper recipe** (a defined process and standards) and **tasting along the way** (reviews and tests). You cannot sprinkle quality on at the end - it must be **baked in from the start**.

Example: A team with clear requirements, trained developers, code reviews and automated tests delivers fewer defects than a team that only 'tests at the end'.

ANSWER - write this in the exam (draw the figure too)

Definition: Software quality is the degree to which software **meets its requirements and the customer's expectations** (conformance to requirements and fitness for use) and shows desired attributes such as reliability, usability, efficiency and maintainability.

Parameters (factors) required for good quality

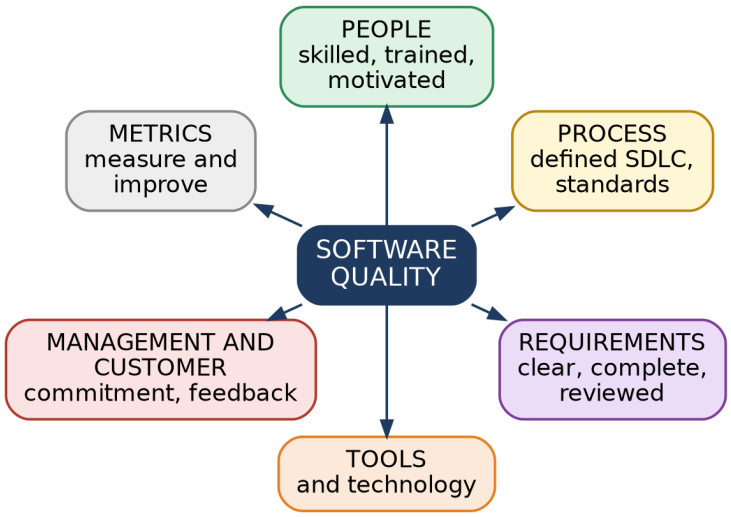


Figure: Parameters that decide software quality

People (skills, training, motivation); **Process** (defined SDLC, standards, procedures); **Requirements** (clear, complete, stable); **Tools and technology** (test tools, configuration management, defect tracker); **Management and customer** (commitment, resources, customer involvement and feedback); **Metrics** (measure defects, effort, coverage to improve). Adequate time and budget are also needed.

Facts about achieving software quality

Fact	Meaning
Quality is built in, not tested in	Testing finds defects but cannot create quality; good process prevents them
Early defect removal is cheaper	A defect costs far more to fix in later phases
Most defects start in requirements and design	So reviews of early work products give the biggest gain
Quality is everyone's job	Developers, testers, managers and customers all share responsibility
Testing cannot prove absence of defects	Exhaustive testing is impossible; testing shows only that defects exist
Management commitment is essential	Without time, budget and support, quality efforts fail
What is not measured cannot be improved	Metrics and records are needed for improvement
The customer defines quality	Software must satisfy user needs, not only specifications

Activities to achieve high software quality



Figure: Quality activities across the project

- **Plan:** prepare the SQA plan, define quality goals, standards and procedures.
- **Build right:** manage requirements, follow design and coding standards, use configuration and change control.
- **Review:** technical reviews, inspections and walkthroughs of requirements, design and code (verification).
- **Test:** unit, integration, system and acceptance testing (validation); regression testing after changes.
- **Measure and audit:** collect metrics; audits to confirm the process is followed.
- **Improve:** defect tracking, root cause analysis, corrective and preventive actions, training and risk management.

HARD WORDS EXPLAINED - write the ones you need

Conformance to requirements: product does exactly what was specified

Fitness for use: product is actually useful for the customer's purpose

Verification / validation: checking we build the product right (reviews) / the right product (testing)

Configuration management: controlling versions and changes of code and documents

Regression testing: re-testing after changes to ensure nothing broke

Root cause analysis: finding the real reason behind a defect to stop it recurring

FINAL TAKEAWAY: Good software quality comes from the right people, process, requirements, tools and metrics, and from planned activities - review, test, measure, improve - done from the start.

Q3. Quality Control Tools: Ishikawa's 7 Basic Tools, Charts and Comparisons

Covers 6 Qs | 4-9 marks

This one answer covers these questions: *1) Explain Ishikawa's basic tools for quality control. 2) Illustrate Ishikawa's 7 basic tools in detail. 3) Explain in brief: histogram, flowchart, control chart. (Part A) | 4) Compare run charts and control charts. 5) Clarify flowchart, run charts and control charts. (Part B) | 6) Compare Ishikawa's (cause-and-effect / fishbone) diagram and histogram. (Part C)*

BASIC IDEA - just to understand (no need to write)

These seven tools are the **quality toolbox** of a technician: simple pictures that turn raw data into answers. A **flowchart** shows *how* the work flows, a **check sheet** collects data, a **histogram** shows how data is spread, a **Pareto chart** shows the vital few causes, a **fishbone** lists possible causes, a **scatter diagram** shows if two things are related, and a **control chart** shows if the process is stable.

Example: A team wants to cut post-release defects: check sheet records defects, Pareto shows 'requirement errors' cause most, fishbone finds why, control chart then monitors improvement.

ANSWER - write this in the exam (draw the figure too)

Part A: Ishikawa's seven basic tools of quality control

Proposed and popularized by **Kaoru Ishikawa** (Japan), these seven simple graphical tools are used to collect, analyse and present quality data and to solve most quality problems.

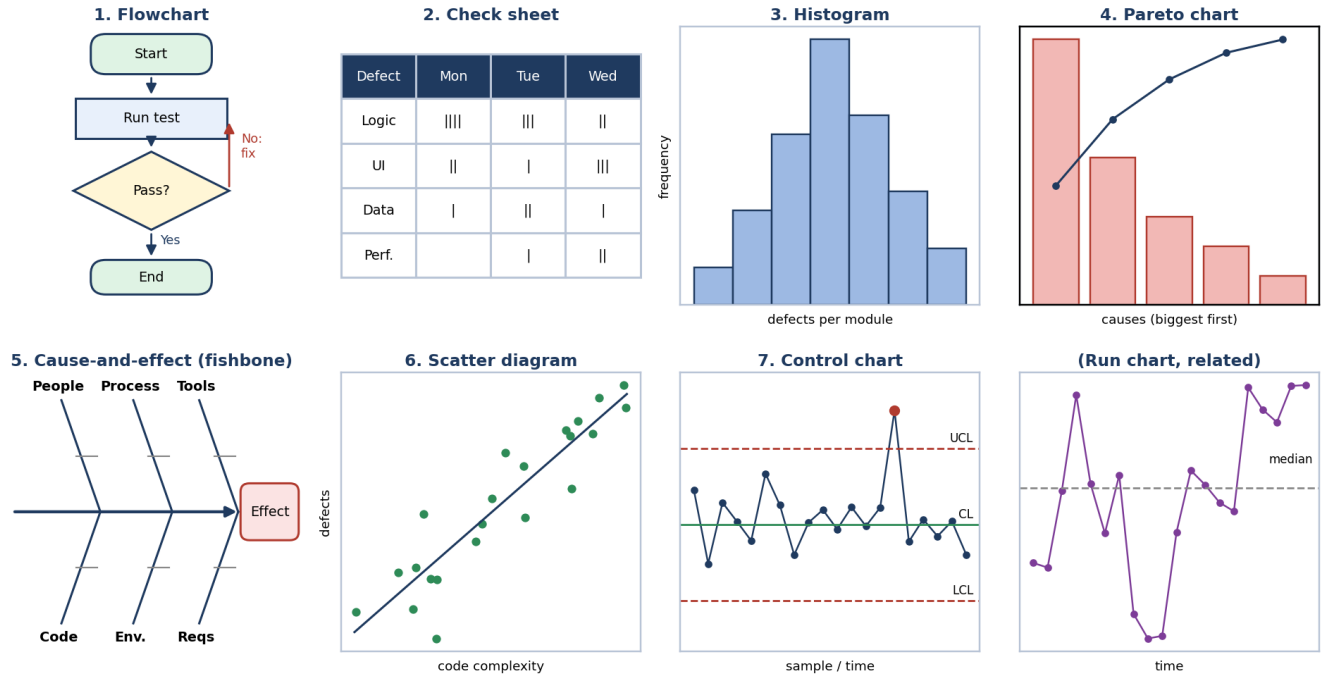


Figure: The seven basic QC tools at a glance (run chart shown as a related tool)

Tool	What it is / purpose	Software example
1. Flowchart	Diagram of the steps of a process using symbols (start/end, process, decision) to understand and improve it	Flow of defect reporting and fixing
2. Check sheet	Simple form to collect data in a structured way using tally marks	Count of defects by type per day
3. Histogram	Bar chart of frequency distribution; shows spread and shape of data	Number of modules by defect count
4. Pareto chart	Bars in descending order plus cumulative line; 80/20 rule: few causes give most problems	20% of modules cause 80% of defects
5. Cause-and-effect (Ishikawa / fishbone)	Brainstorming diagram that groups possible causes of one effect	Why do defects escape to production?
6. Scatter diagram	Plots two variables to see if they are related (correlation)	Code complexity vs defects
7. Control chart	Plots data over time with centre line and upper / lower control limits to check process stability	Defects per build over time

Some books list stratification (splitting data into groups) or the run chart in place of the flowchart; the idea is the same.

Part B: Flowchart vs run chart vs control chart

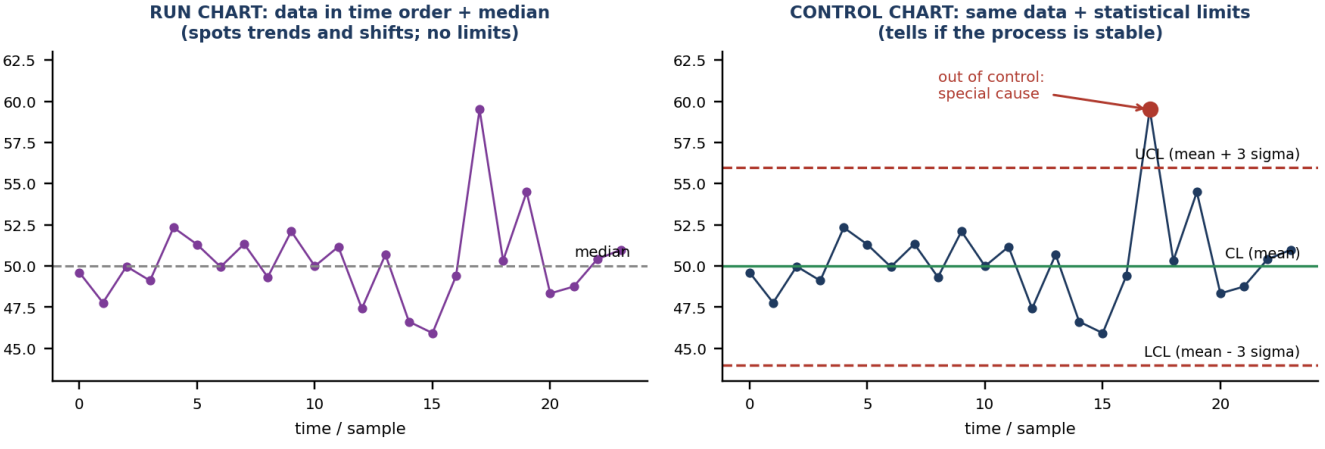


Figure: Run chart and control chart of the same data: only the control chart has limits

Basis	Flowchart	Run chart	Control chart
What it shows	Sequence of steps of a process	Data plotted in time order	Data in time order with statistical limits
Type	Process diagram (no data)	Simple data chart	Statistical data chart
Lines	Arrows between symbols	Centre line = median (or average)	Centre line (mean), UCL and LCL (mean +/- 3 sigma)
Purpose	Understand and improve the process	Spot trends, shifts, patterns over time	Decide if the process is in control or out of control
Detects	Bottlenecks, extra steps	Trends and shifts only	Special-cause variation (points outside limits, unusual patterns)
Complexity / use	Easiest; used at design of process	Simple; used early, small data	Needs calculations; used to monitor mature processes
Example	Defect-fixing workflow	Defects per week	Defect density per release with limits

Part C: Ishikawa's (fishbone) diagram vs histogram

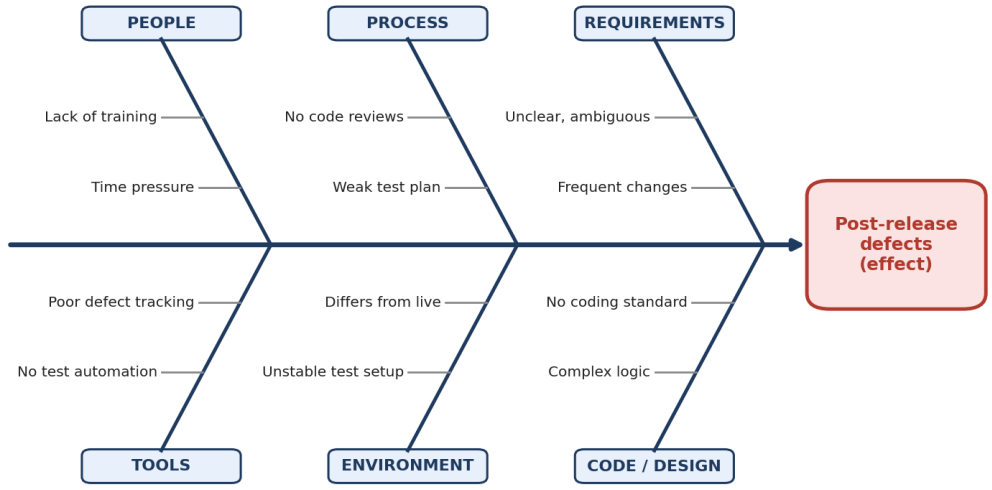


Figure: Ishikawa (fishbone) diagram for post-release defects

Basis	Ishikawa / cause-and-effect diagram	Histogram
Purpose	Find the possible CAUSES of one problem	Show HOW DATA is distributed (frequency)
Question answered	Why is this happening?	How often, and how spread out?
Data type	Qualitative: ideas from brainstorming	Quantitative: measured numbers
Structure	Fish skeleton: effect at head, cause categories as bones (people, process, tools, requirements, environment, code)	Bars touching each other; x-axis = value ranges, y-axis = frequency
Used when	Analyzing root cause, in a team	Analyzing data pattern, variation, average and shape
Software example	Why do defects escape to production?	Number of modules with 0-2, 3-5, 6-8 defects

Note: Ishikawa's diagram is the fishbone diagram; questions that say 'Ishikawa's flowchart' mean this cause-and-effect diagram.

HARD WORDS EXPLAINED - write the ones you need

Variation: difference in results each time a process runs

UCL / LCL / CL: Upper Control Limit / Lower Control Limit / Centre Line (mean) on a control chart

In control: only normal random variation; process is stable and predictable

Special cause: an unusual cause that makes a point fall outside the limits

Correlation: two things rising or falling together

Pareto principle (80/20): about 80% of effects come from 20% of causes

Frequency distribution: how many times each value or range occurs

FINAL TAKEAWAY: The seven QC tools turn data into decisions: flowchart (process), check sheet (collect), histogram (spread), Pareto (vital few), fishbone (causes), scatter (relationship), control chart (stability).

Q4. SQA Plan, Tasks-Goals-Metrics and Defect Removal Effectiveness

Covers 3 Qs | 4-9 marks

This one answer covers these questions: *1) Write a note on the Software Quality Assurance (SQA) plan. (Part A) | 2) What are the different tasks, goals and metrics in SQA? (Part B) | 3) What do you understand by defect removal effectiveness? (Part C)*

BASIC IDEA - just to understand (no need to write)

SQA is the team of 'quality referees' of a project. The **SQA plan** is their **game plan**: what will be checked, by whom, when and by what rules. **Tasks** are what they do, **goals** are why they do it, and **metrics** are the numbers that show whether the goal was reached. **Defect removal effectiveness** is the score: of all defects, what percentage did we catch before the customer did?

Example: Task: audit the design. Goal: good design quality. Metric: average complexity of modules. DRE: 90 defects caught by us, 10 by customer = 90%.

ANSWER - write this in the exam (draw the figure too)

Part A: SQA plan

Definition: The **SQA plan** is a document (based on **IEEE 730**) that states **how software quality assurance will be carried out for a project**: its organization, tasks, standards, reviews, audits, tests, tools and records.

Section of SQA plan	What it contains
Purpose and scope	Why the plan exists and which software and activities it covers
Management	SQA organization, roles and responsibilities, tasks
Standards, practices, conventions	Coding, documentation and process standards to follow
Reviews and audits	What reviews, inspections and audits are done and when
Testing	Test strategy and its link to quality goals
Problem reporting and corrective action	How defects and non-conformances are reported, tracked and closed
Tools, techniques, metrics	Tools used and measures collected (defect density, coverage)
Records, training, risk	Records kept, training needs, quality risks and how they are handled

Part B: Tasks, goals and metrics in SQA

SQA tasks: prepare the SQA plan; take part in developing the project's process; review engineering activities to check process compliance; audit work products; ensure deviations are documented and handled; record and report non-compliance; coordinate change and defect control.

Goal	Attributes checked	Metrics
Requirements quality	Ambiguity, completeness, volatility, traceability	Number of ambiguous statements, % requirements incomplete (TBD), requirement change rate
Design quality	Architectural integrity, component and interface complexity	Cyclomatic complexity, coupling and cohesion, fan-in / fan-out
Code quality	Complexity, maintainability, understandability, documentation	Cyclomatic complexity, lines per module, comment ratio, defect density
Quality control effectiveness	How well reviews and tests find defects	Defects per review hour, test coverage %, defect removal effectiveness

Part C: Defect Removal Effectiveness (DRE)

Definition: DRE measures the ability of the quality activities (reviews, testing) to **remove defects before the software is delivered**. **DRE = E / (E + D) x 100%**, where **E** = defects found before delivery and **D** = defects found by the customer after delivery. The ideal value is 100%.

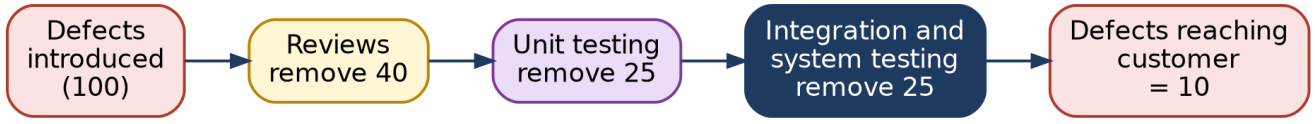


Figure: Defects are filtered by each activity; DRE = 90 / (90 + 10) = 90%

Example: E = 40 + 25 + 25 = 90 defects found internally, D = 10 reached the customer, so DRE = 90/100 = **90%**. DRE can also be calculated phase-wise (defects found in a phase / defects present in it) to show how many defects each phase **contains**. **Use:** a high DRE means fewer customer-found defects, lower rework cost and better quality; improve it with early reviews, inspections, better test cases and root cause analysis.

HARD WORDS EXPLAINED - write the ones you need

Audit: independent check that the process and work products follow the plan and standards

IEEE 730: an international standard describing SQA processes and plans

Cyclomatic complexity: a number showing how many independent paths a program has; higher = more complex

Coupling / cohesion: how much modules depend on each other / how focused one module is

Defect density: defects per KLOC (thousand lines of code)

Volatility: how frequently requirements change

FINAL TAKEAWAY: The SQA plan is the project's quality game plan; tasks, goals and metrics show what is done and measured; DRE tells what percentage of defects we caught before release.

Q5. Total Quality Management (TQM)

Covers 1 Q | 6-9 marks

This one answer covers these questions: *1) Explain Total Quality Management in detail.*

BASIC IDEA - just to understand (no need to write)

In a good restaurant, the chef, waiter, cleaner and owner **all** feel responsible for the guest's experience, and they **keep improving** every day. That is TQM: **Total** = everyone, **Quality** = meeting customer needs, **Management** = a continuing way of running the organization.

Example: A software company where developers, testers, support and managers all take part in reviews, collect customer feedback and improve the process every release.

ANSWER - write this in the exam (draw the figure too)

Definition: Total Quality Management (TQM) is a management approach for long-term success through **customer satisfaction**, in which **all members of the organization participate** in continually improving processes, products, services and culture. It grew from the work of **Deming, Juran, Crosby and Ishikawa**.

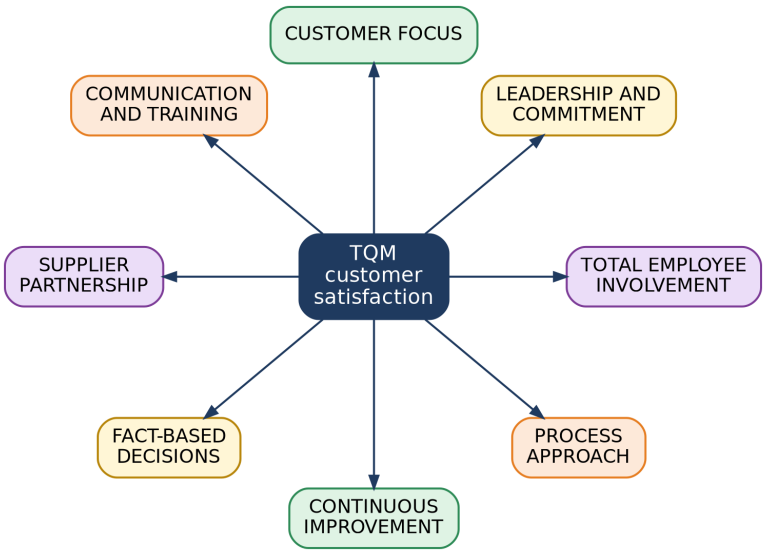


Figure: Principles of Total Quality Management

Principle	Meaning in software
Customer focus	Customer decides quality; collect and act on user feedback
Leadership and commitment	Management sets quality policy and provides resources
Total employee involvement	Everyone owns quality; teams, empowerment, quality circles
Process approach	Manage work as processes; remove defects at the source
Continuous improvement	Kaizen / PDCA cycle: Plan - Do - Check - Act, repeated
Fact-based decisions	Use metrics and QC tools (histogram, Pareto, control charts)
Supplier partnership	Work with vendors and tool providers as quality partners
Communication and training	Share goals openly; train people in quality methods

TQM in software: defect prevention, reviews and inspections, standards, metrics, customer feedback and process improvement across the SDLC; it covers QA and QC. **Benefits:** higher customer satisfaction, lower rework and cost, better morale and competitiveness. **Challenges:** needs time, culture change, training and strong top-management commitment.

HARD WORDS EXPLAINED - write the ones you need

Kaizen: Japanese word for continuous small improvements

Empowerment: giving employees authority to make quality decisions

PDCA: Plan-Do-Check-Act: repeating improvement cycle (Deming cycle)

Deming / Juran: quality pioneers whose ideas form the base of TQM

Quality circle: small group of employees who regularly meet to solve quality problems

Defect prevention: stopping defects from happening rather than finding them later

FINAL TAKEAWAY: TQM makes quality everyone's responsibility: customer focus, involvement of all, process thinking and continuous improvement based on facts.

Q6. Software Maintenance and Maintaining SQA

Covers 2 Qs | 4-9 marks

> ***This one answer covers these questions:*** *1) What do you understand by software maintenance? Explain in detail. (Part A) | 2) Can you explain how to maintain SQA? (Part B)*

BASIC IDEA - just to understand (no need to write)

Part A: Software is like a car after purchase: it needs **repairs** (bugs), **adjustments** for new roads or fuel (new OS, laws), **upgrades** (new features) and **servicing** (cleaning up the code). **Part B:** The SQA system itself also needs regular servicing, or it becomes outdated and ignored.

Example: A banking app is released. Later: a bug in interest calculation is fixed (corrective), RBI rule change is added (adaptive), a faster search is added (perfective), messy code is cleaned (preventive).

ANSWER - write this in the exam (draw the figure too)

Part A: Software maintenance

Definition: Software maintenance is the **modification of a software product after delivery** to correct faults, improve performance or other attributes, or adapt it to a changed environment. It is the longest phase of the SDLC and often takes more than half of the total life-cycle cost.

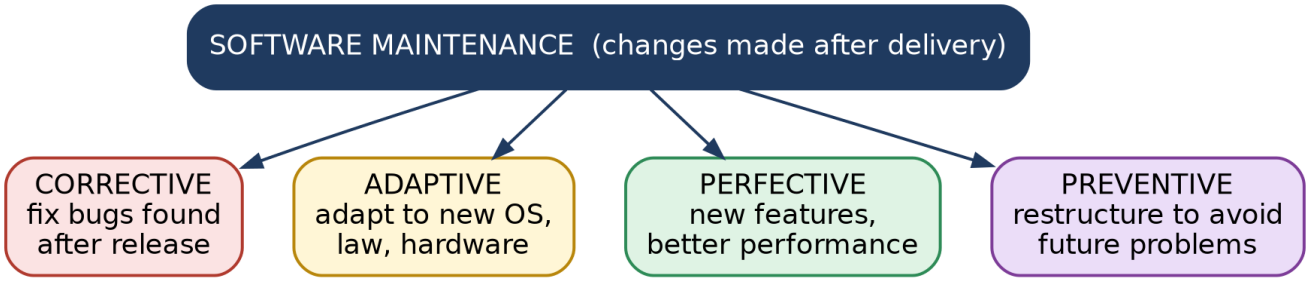


Figure: Four types of software maintenance

Type	Purpose	Example
Corrective	Fix defects found after release	Fix wrong interest calculation
Adaptive	Adapt software to a changed environment	Support new OS version or new tax rule
Perfective	Add features or improve performance / usability (usually the largest share)	Add faster search, new report
Preventive	Restructure or document code to prevent future problems	Refactor complex module

Why needed: defects remain after release, user needs and business rules change, technology and platforms change. **Process:** change request, impact analysis, design the change, implement, **regression test**, release and update documents. **Challenges:** poor documentation, original developers gone, ripple effects of changes, high cost and pressure to fix fast.

Part B: How to maintain SQA

Maintaining SQA means keeping the quality assurance system **effective, current and followed** throughout the project and after delivery, using a continuous improvement cycle.

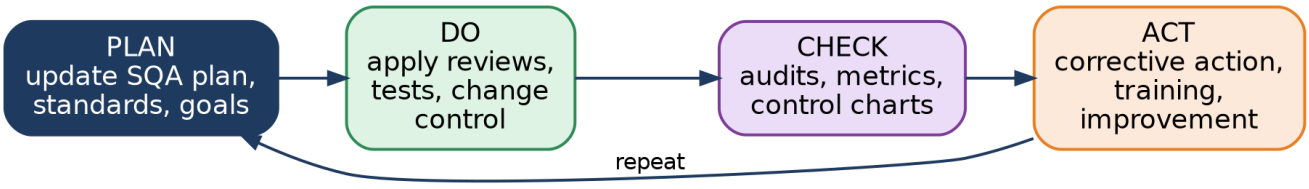


Figure: Keeping SQA alive: PDCA cycle

Activity	How it maintains SQA
Management commitment	Provide budget, time and independent authority for SQA
Update SQA plan and standards	Revise as scope, technology and lessons change

Activity	How it maintains SQA
Regular reviews and audits	Confirm the process is followed; fix non-compliance
Metrics and trend monitoring	Collect defect and process data; use control charts to detect problems early
Defect tracking and root cause analysis	Corrective and preventive actions so defects do not repeat
Training	Keep team skilled in standards, tools and methods
Change and configuration control	Control every change during maintenance; regression testing
Customer feedback and process assessment	Use feedback and CMM / ISO assessments to improve

HARD WORDS EXPLAINED - write the ones you need

Maintenance: changes made to software after it has been delivered

Refactoring: cleaning and restructuring code without changing its behaviour

Regression testing: re-testing to ensure a change did not break existing features

Impact analysis: studying which parts of the system a change will affect

Ripple effect: a small change causing unexpected problems elsewhere

Change request: formal request to modify the software

> **FINAL TAKEAWAY:** Maintenance (corrective, adaptive, perfective, preventive) keeps delivered software useful, and SQA must be maintained through updated plans, audits, metrics and continuous improvement.